Natural Computing

Informatica 2.2 – Computational Intelligence

A.Y. 2025-2026

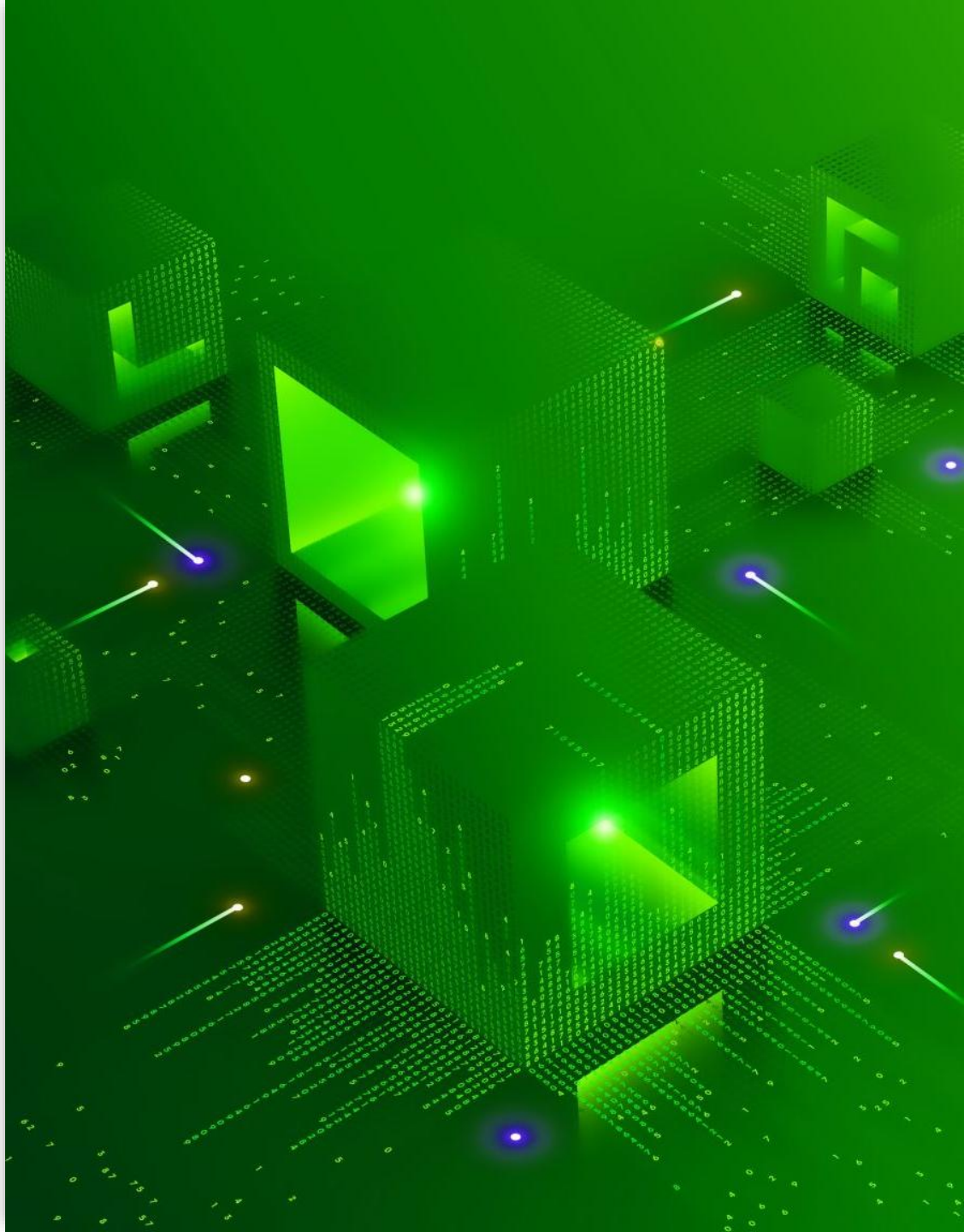
Marco S. Nobile

marco.nobile@unive.it

Natural Computing (NC)?

NC (aka Natural Computation) encompasses three classes of methods:

1) Methods taking inspiration from Nature (e.g., evolutionary algorithms, swarm intelligence, artificial neural networks)
2) Methods using computers to synthesize natural phenomena
3) **Methods that employ natural materials to perform computation**



Outline

- P vs NP
- NP-complete problems
- Turing Machines
- Deterministic vs Non-Deterministic Turing Machines
- DNA computing
  - Adleman molecular computer
  - Wang Tiles
  - Programmable DNA machines
- DNA Origami and DNA bricks

P

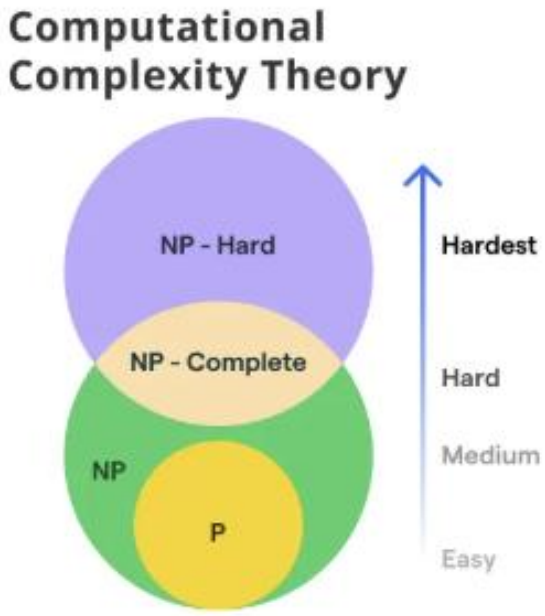
- **Computational complexity** is the amount of **resources** (time, or space) **needed to run an algorithm**
  - One important class is called «P»

- Informally, P contains all (decision) problems that can be **solved efficiently**
  - By efficiently we mean that it is solved by using an amount of computation time that is **polynomial with respect to the input**
  - By **decision problem** we mean all problems that can be posed as a **Y/N question**

- A lot of common problems belong to **P**

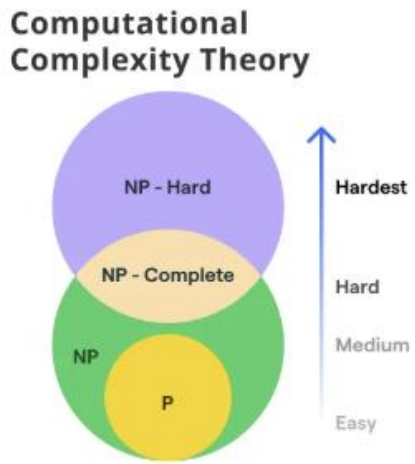
NP

- There are decision problems that **do not have an efficient algorithm, yet**
- Many of them belong to the **NP class of complexity**
- It is not know, at the moment, whether $P \neq NP$
  - Most computer scientists **think they are not equal**, but there is **no proof**
  - It is one of the **Millenium Prize Problems**, if you find a demonstration you win 1 million USD



NP-complete problems

- **Decision problems** belonging to the **NP class of complexity**
- Some of the **hardest in computer science**
- Intractable: **no algorithms are known**… at least on common computers (i.e., **deterministic Turing machines**)
- **Exact solutions** can only be found using a **brute force** approach: **enumerate and evaluate all possible solutions** (usually, exponential or worse)
- **Reducibility**: any NP-complete problem can be transformed into another NP-complete problem (in polynomial time)
- Examples of NP-complete problems: TSP, knapsack, SAT, graph coloring, SubsetSum, Hamiltonian Path…

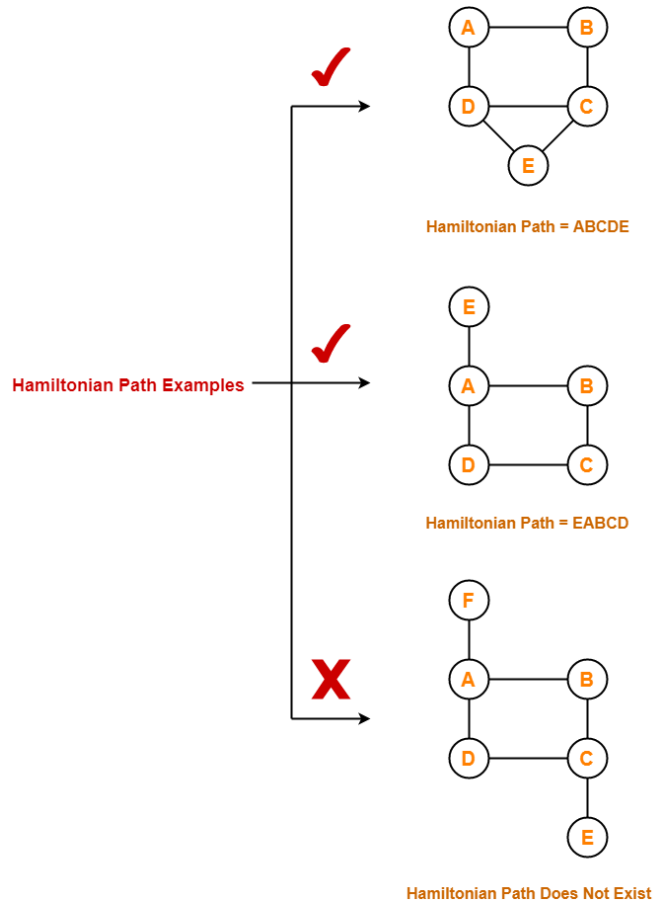


NP-C: Hamiltonian Path Problem (HPP)

- A path on a graph is Hamiltonian if it **starts at a specific vertex**, **visits every vertex exactly once**, and **returns to the first vertex**

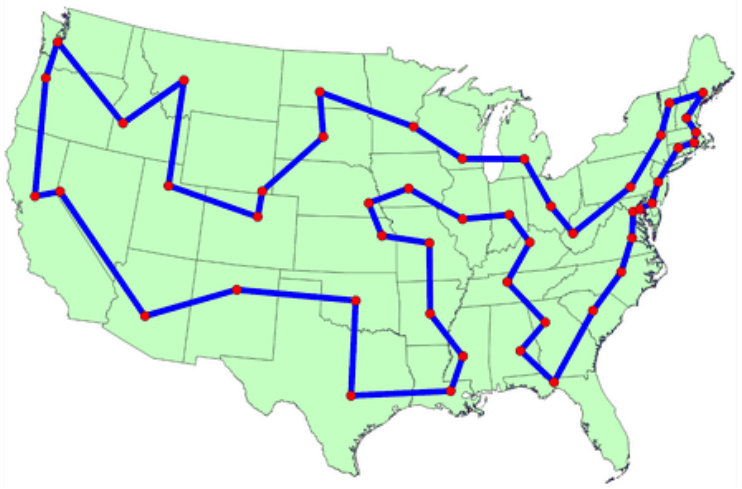
- **HPP: decide if a graph contains a Hamiltonian path**

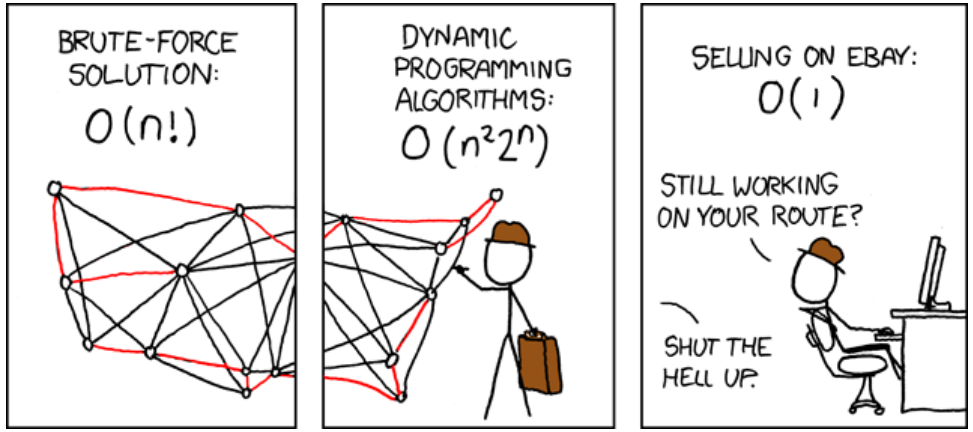
- **NP-complete**: no efficient algorithm is known, we must **enumerate all possible paths**
  - Alas, the number of paths is **factorial** in the number of vertices, that is, $\mathcal{O}(n!)$
  - **Impractical** even for small instances on conventional computers



NP-C: Travelling Salesman Problem (TSP)

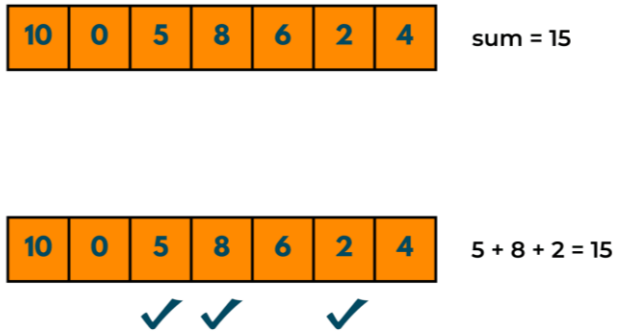
- The salesman must **travel to all cities exactly once** before returning home
- **The distance** between neighbouring cities **is given**, and it is assumed to be **identical in both directions**
- Goal: **minimize the total distance** to be travelled
- The **decision variant** of TSP is NP-Complete





NP-C: SubsetSum

- Assume a **sequence of numbers** B and a **value** v
- Find the subset of B such that the **sum** of the numbers equals v
- The decision variant («is there a subset…») is NP-complete



One interesting characteristics of NP problems

- To *find* a solution, in the worst case, **we must test all possible cases**, which often requires to explore a **huge solution space**

- To *check* whether a solution is valid (or not) is **much easier and faster**: we just **evaluate the solution** (in polynomial time)
  - E.g., in the TSP we just sum the distances between the cities; in Subset Sum we add the numers in the solution, etc.

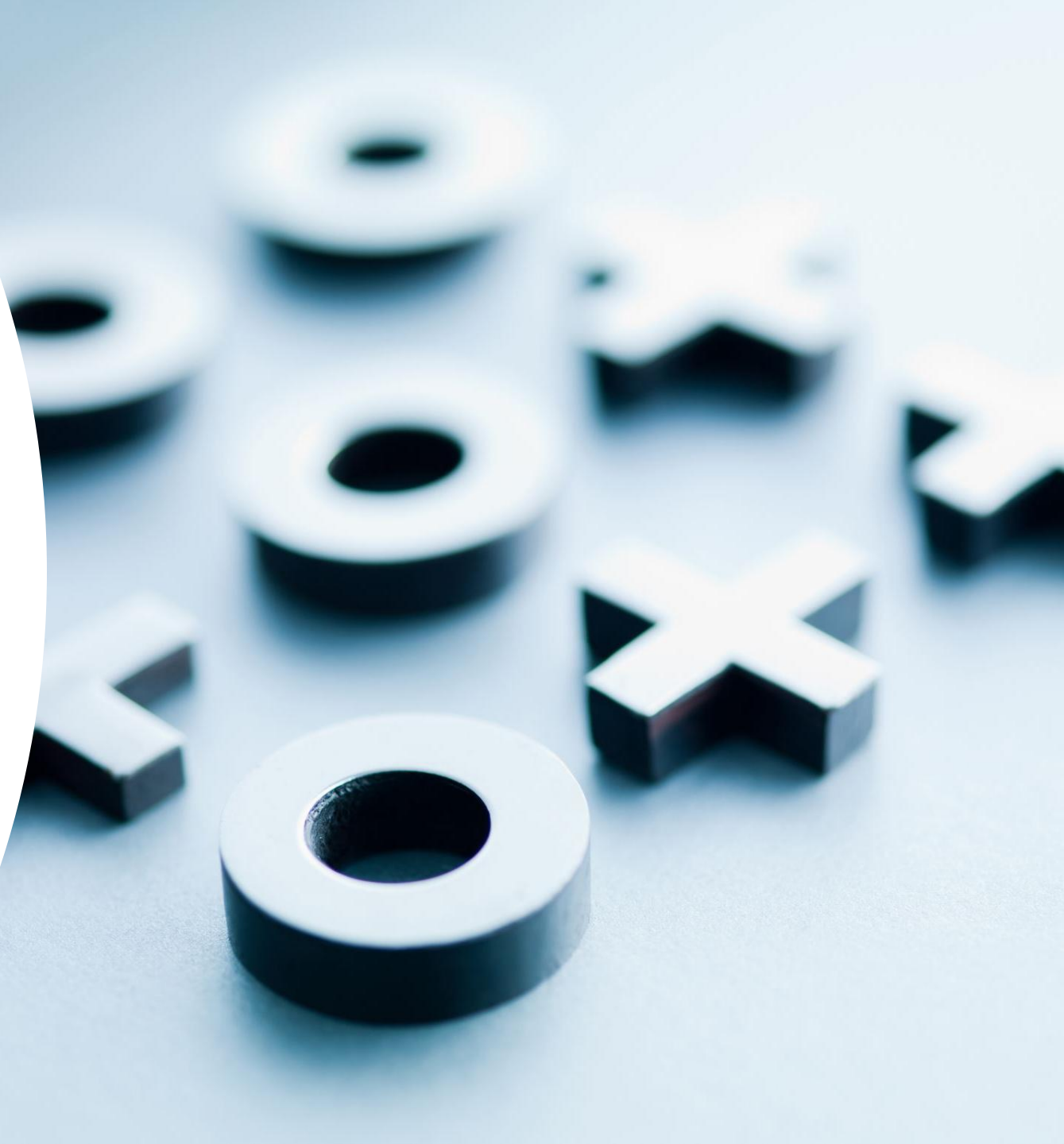
- Thus, an algorithm that solves an NP problem in polynomial time must be somehow able **to explore all possibile solutions at once** (!)

One key concept

NP does not mean «non polynomial complexity»

NP actually means **«polynomial on a *non-deterministic* Turing machine»**

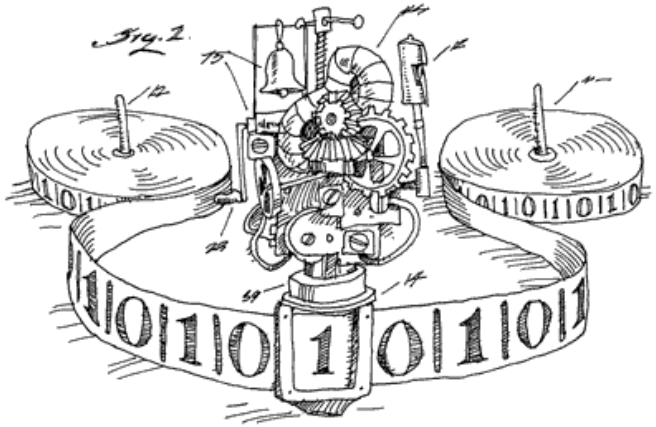
That is, a **special type of Turing machine**



Turing Machines [1/2]

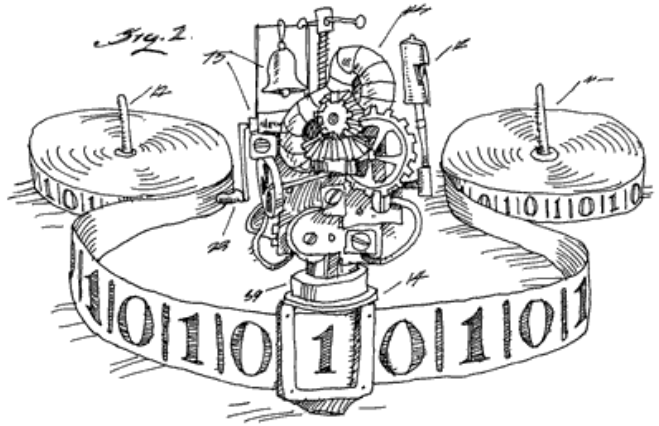
A (deterministic) Turing Machine (DTM) is composed of three parts:

- **A tape**, organized in **cells**, that can carry **symbols** (from a finite alphabet)
- **A head** over the tape. The head can **read** or **change** the current symbol on the tape, and can **move the tape** one cell to the left or right
- **A state**, selected from a finite set of states



Turing Machines [2/2]

- At each step, the head **reads the symbol** on the tape
- Based on the **read symbol** and the **current state**, the DTM can **write a symbol** in that cell and can **move the head** (← or →) or **halts computation**
- **The choice** about what to write on the tape and whether to move or halt is based on a **finite table** that specifies the action for **all pairs** (state, symbol)

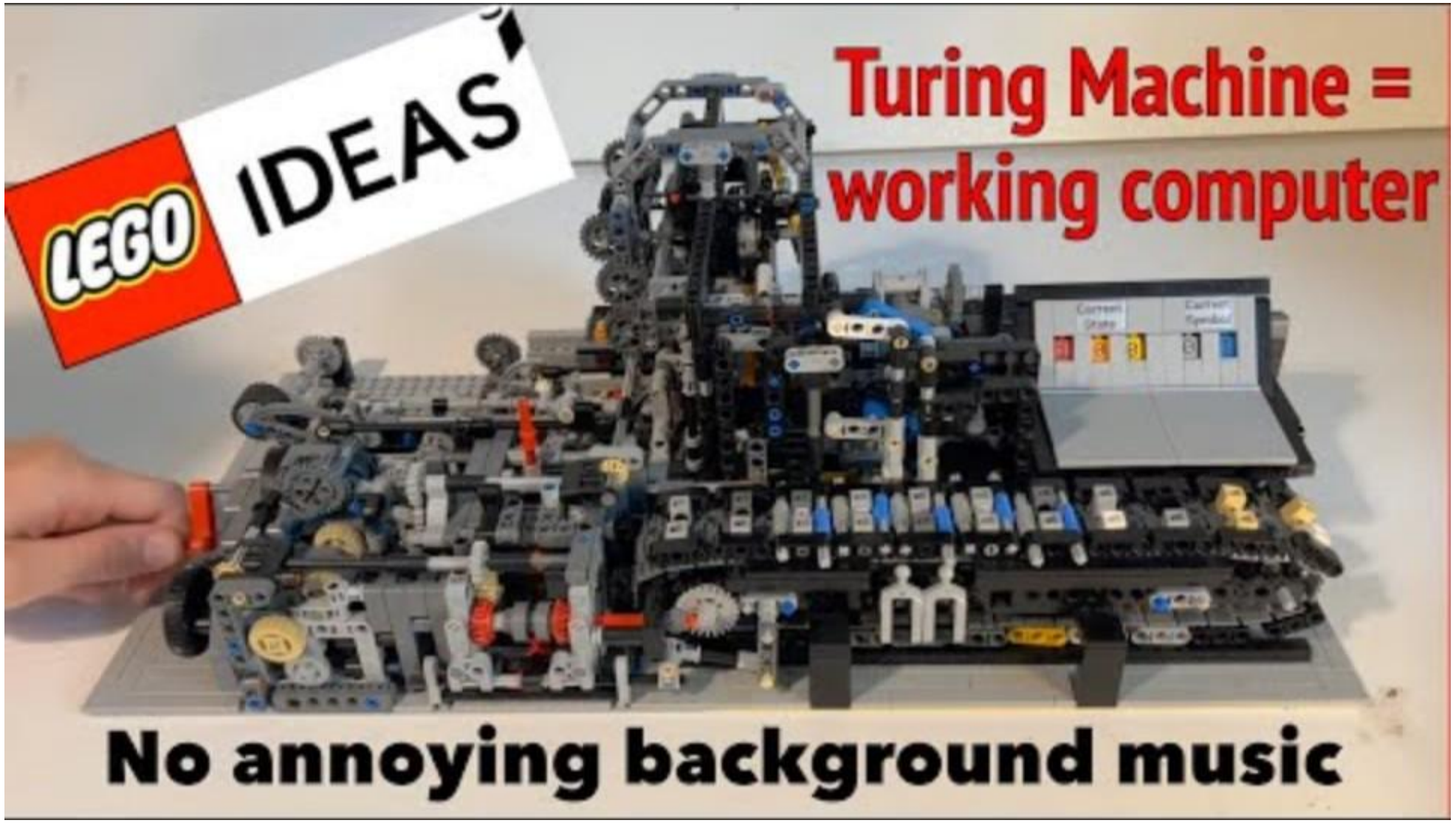


Symbols

States δ	0	1	B
q_0	$(q_0, 0, R)$	$(q_0, 1, R)$	(a_0, B, L)
q_1	–	–	–
a_0	(s_0, B, L)	(s_1, B, L)	(q_1, B, N)
a_1	(s_1, B, L)	(s_2, B, L)	–
a_2	(s_2, B, L)	(s_0, B, L)	–
s_0	(a_0, B, L)	(a_2, B, L)	(q_1, B, N)
s_1	(a_1, B, L)	(a_0, B, L)	–
s_2	(a_2, B, L)	(a_1, B, L)	–

← Example of programming table

LEGO Turing machine



Back to NP-complete problems…

- **Exact solutions** can only be found using a **brute force** approach: **enumerate and evaluate all possible solutions** (usually, exponential or worse)

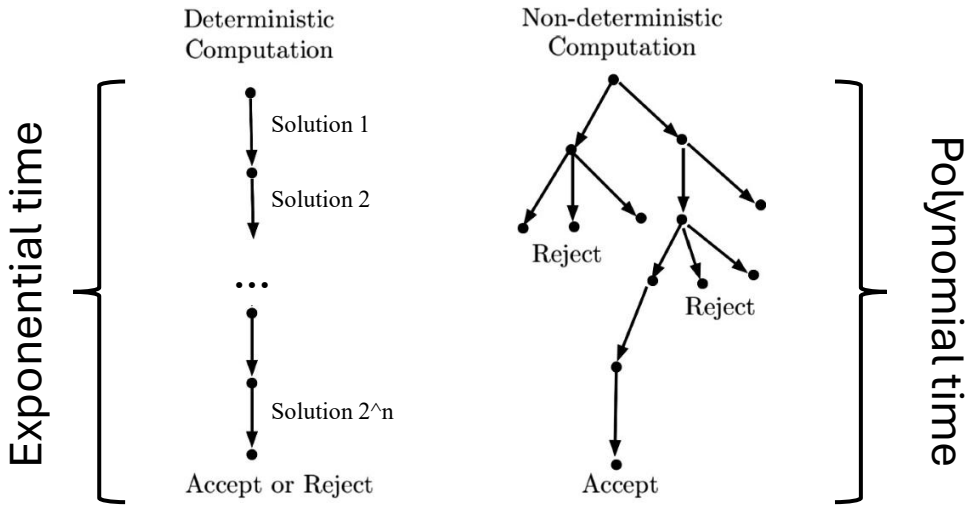
- Thus, on a DTM, we must **generate a large number of possible solutions** and **verify each one of them** (in polynomial time)

- Let us imagine if this was not the case…

Non-Deterministic Turing Machines (NDTM)

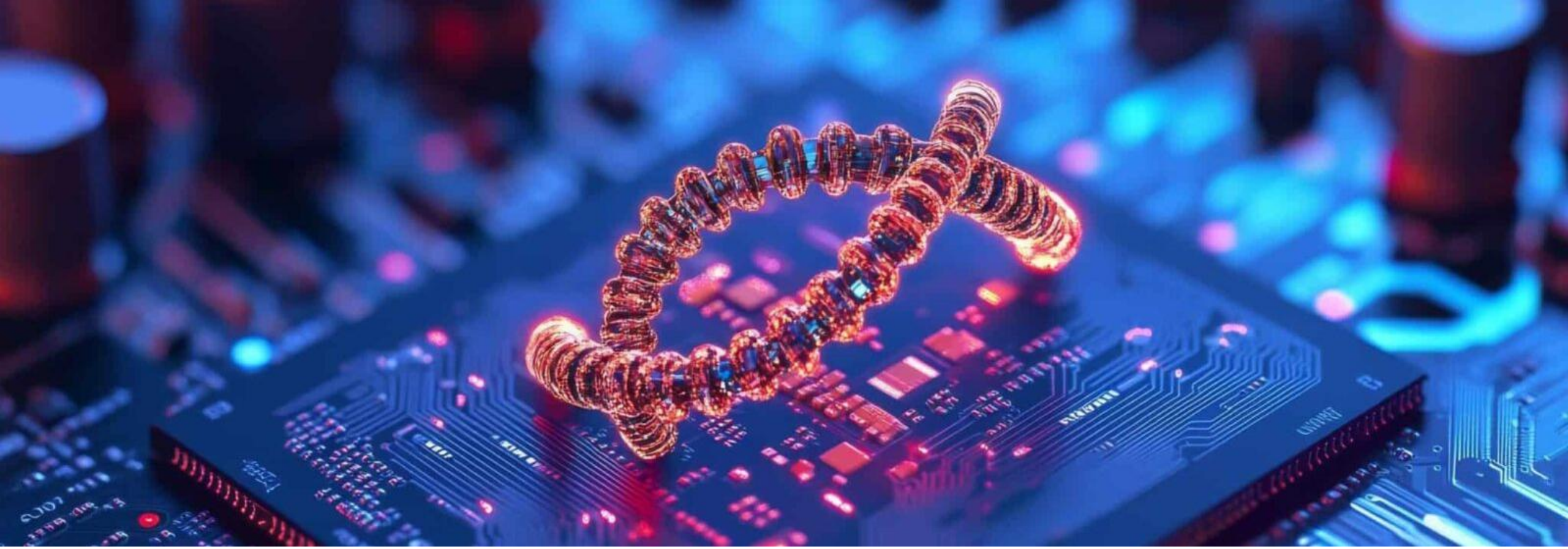
- To get a **solution of an NP problem in polynomial time** the machine must be able to **explore all possible solutions simultaneously**

- Or, as an alternative, we need an **«oracle»** module that gives us a solution



- In both cases we easily **check** the solution in **polynomial time**

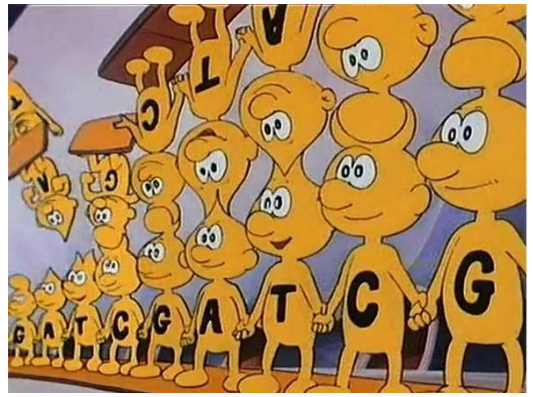
- Nice. HOW?



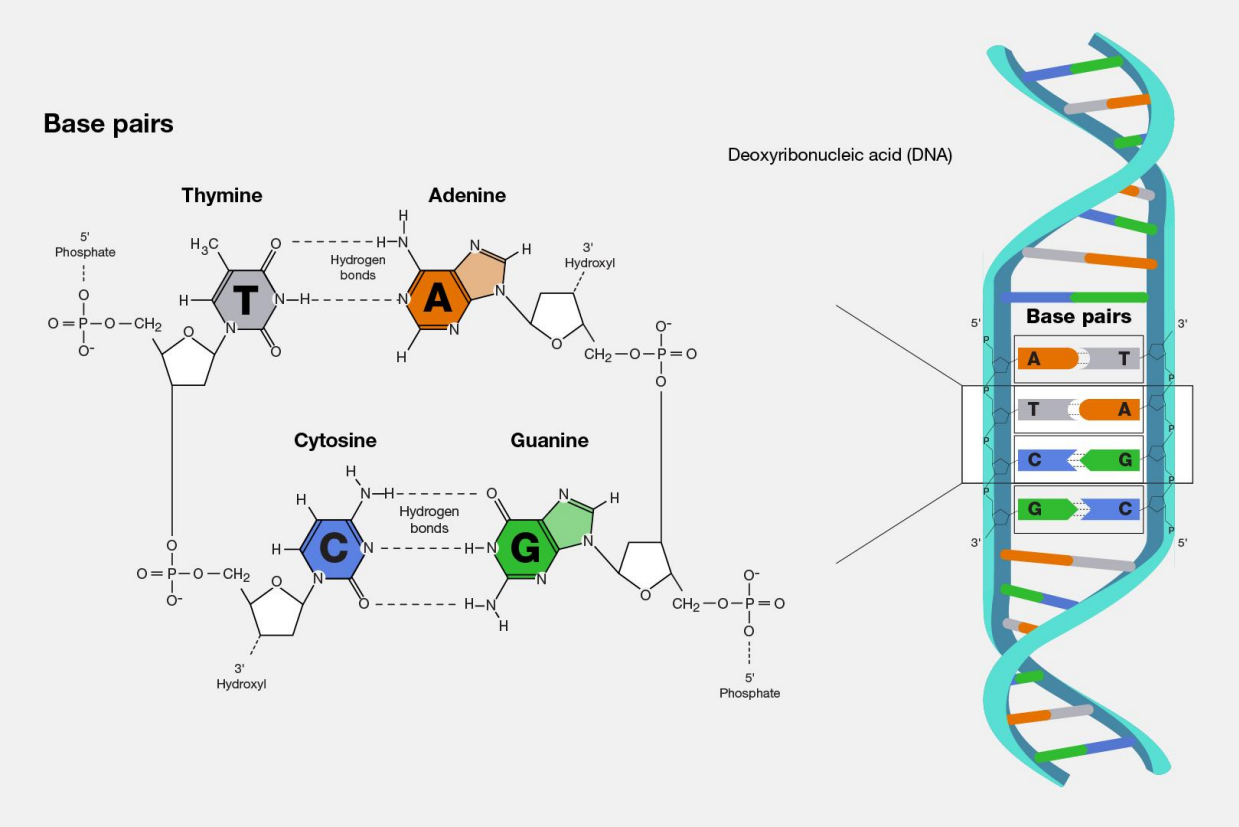
Computing with DNA

DNA computing is based on the idea of **exploiting spontaneous DNA self-assembly** processes to perform computation

DNA

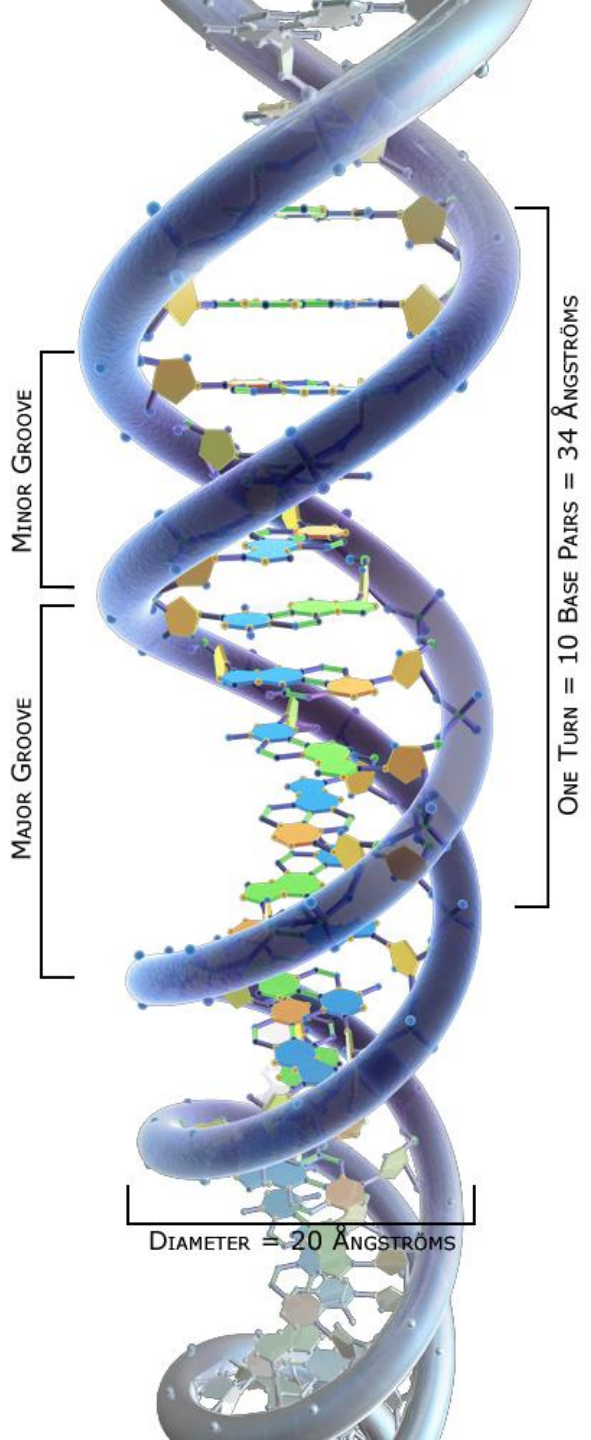


- DNA is made of **two linked strands** that wind around each other
- Each strand has a backbone made **of alternating sugar** (deoxyribose) **and phosphate groups**
- Attached to each sugar is **one of four "bases"**: adenine (A), cytosine (C), guanine (G) and thymine (T)
- **A base pair** consists of **two complementary DNA nucleotides** (A-T, C-G)



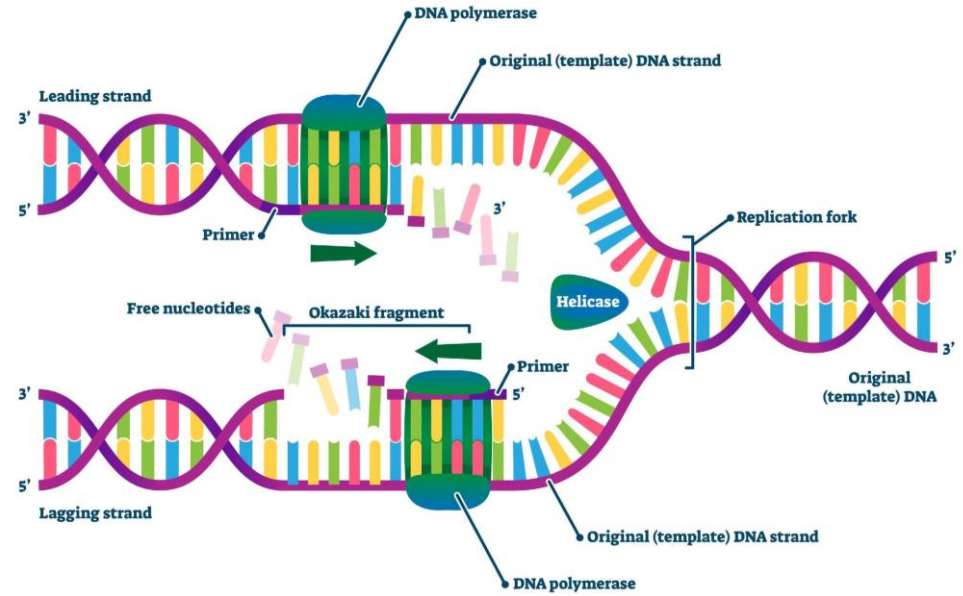
Structural properties of DNA

- DNA is very stable, regular, a rigid crystal: desirable characteristics to **store information** and **build artifacts**
- DNA has a minor and a major groove
- The double helyx makes a complete turn **every 10 base pairs**
- It can be arranged in **regular lattices**
- The diameter is **20 Ångstroms** (2 nm)



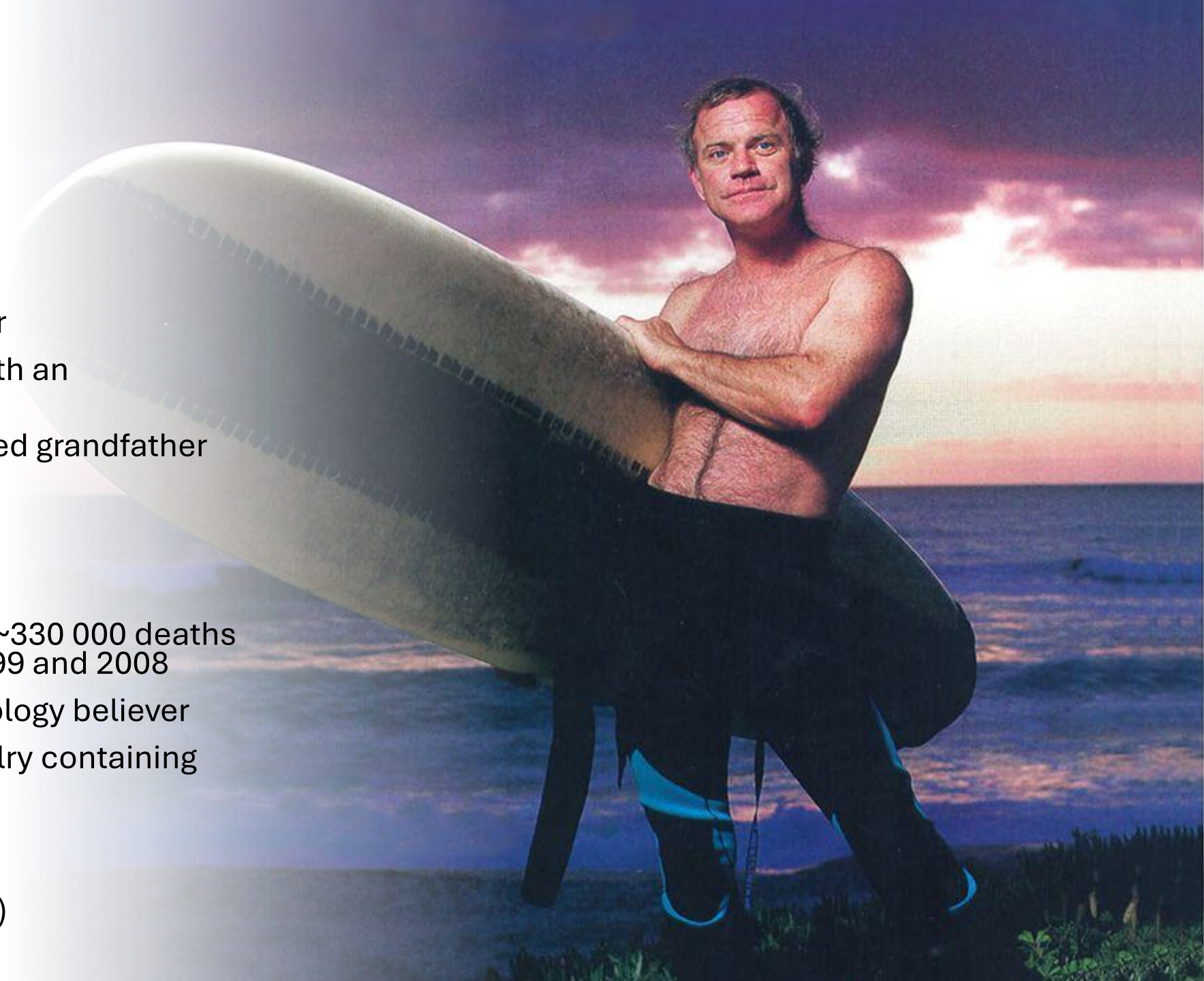
Polymerases

- **DNA polymerases** are **enzymes** that are responsible for **DNA duplication**
- They **read** the code from one strand and form the **complementary sequence** on the other strand by **assembling free nucleotides**
- Polymerase does not start replication on its own but **requires a «primer» sequence**
- Interesting fact: it is **extremely well conserved** across all organisms, it seems irreplaceable



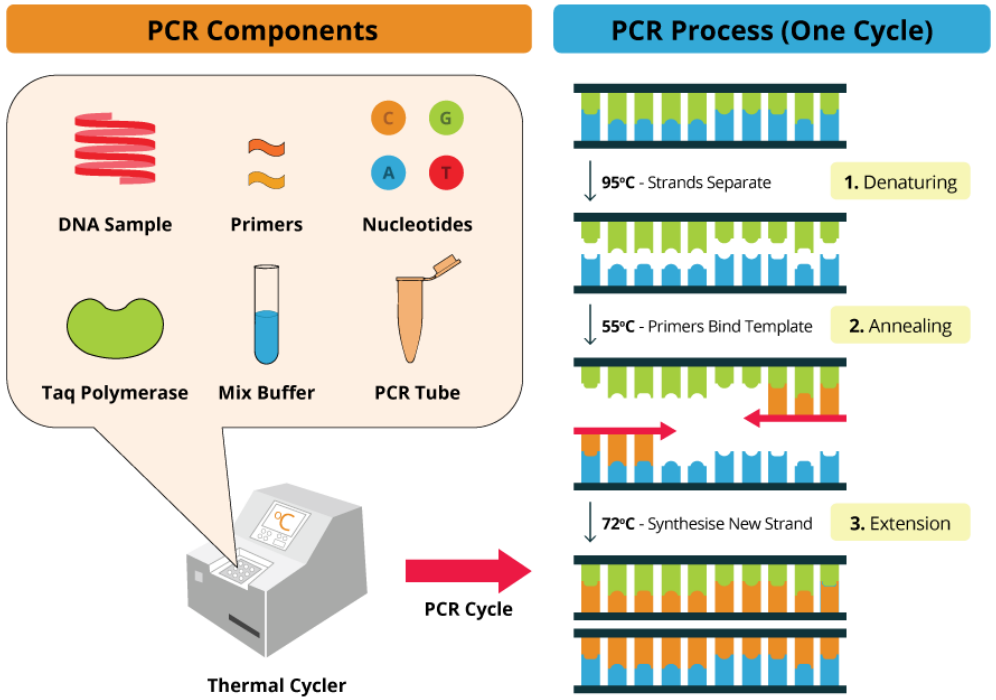
Kary Mullis

- Surfer and guitar player
- Science fiction writer
- LSD producer and fond user
- The first man to ever talk with an alien fluorescent raccoon
- Had a beer with his deceased grandfather
- Former bakery owner
- Climate change denier
- Ozone layer denier
- HIV denier, responsible for ~330 000 deaths in South Africa between 1999 and 2008
- Space traveler and yet astrology believer
- Unauthorized seller of jewelry containing Marylin Monroe's DNA
- **Nobel prize in Chemistry**
- **Inventor of the PCR** (polymerase chain reaction)



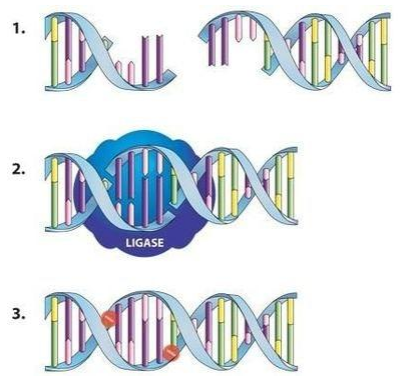
PCR

- **Polymerase Chain Reaction**
- **Amplifies a DNA signal**
- Process:
  - Put the DNA sample, nucleotides and polymerases in a test tube
  - Denaturate the DNA by heating the sample
  - Attach the primers to the region to be amplified
  - The polymerase will bind to the primers and extend the DNA region
  - Rinse and repeat 30x
  - The tube is now filled with the amplified signal marked by primers



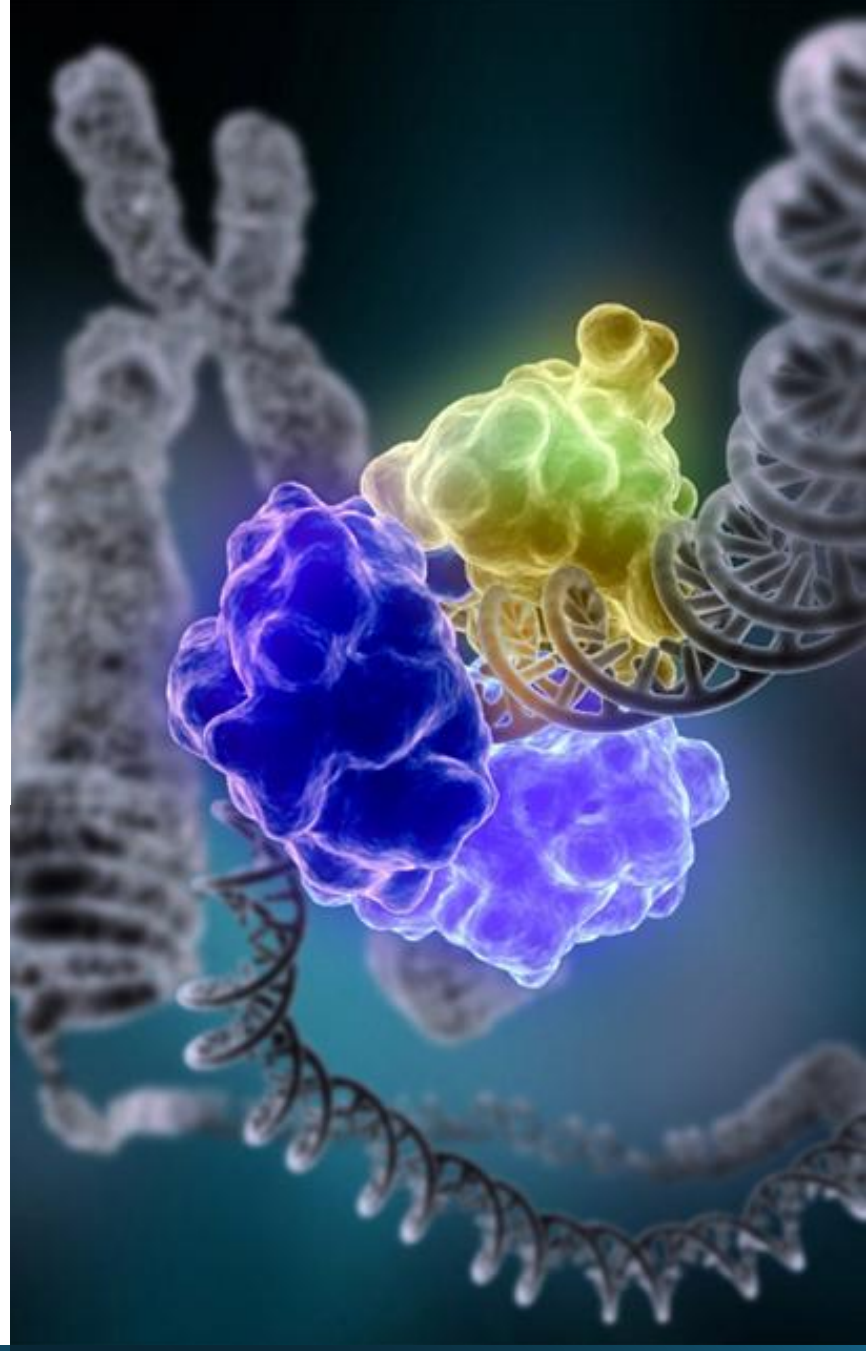
Ligase

- A Ligase is an enzyme that **catalyzes the joining** (ligation) of two molecules



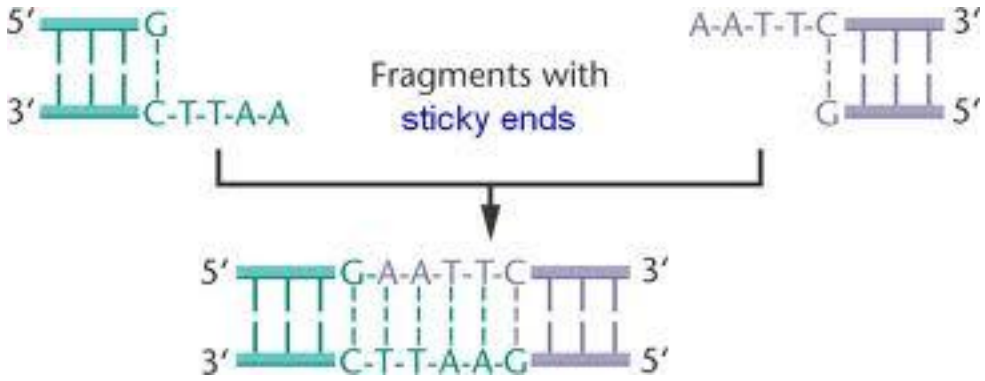
- **DNA ligase** can accelerate the join between **two DNA complementary fragments** by phorming phosphodiesteres bonds

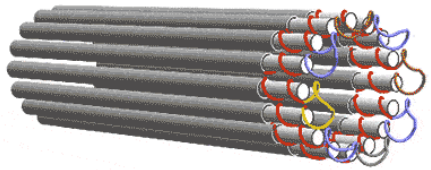
- Such complentary fragments are called **«sticky ends»...**



«Sticky ends»

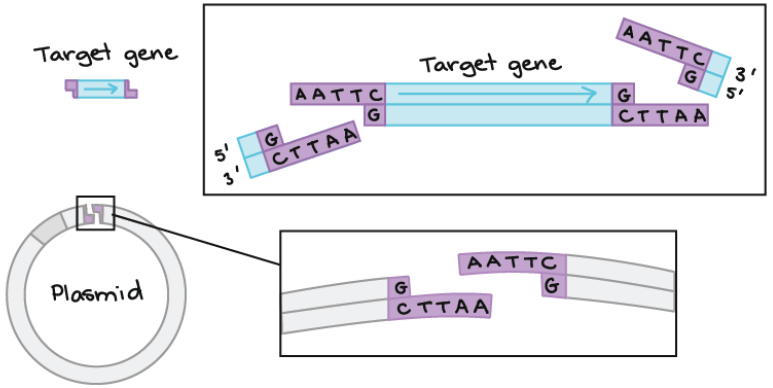
- If two DNA fragments expose **single strands fragments with complementary bases**, they tend to **attach to each other**
  - The process is accelerated by DNA ligases
- This process is known as **DNA self-assembly**
- Such exposed strands are called **sticky ends**





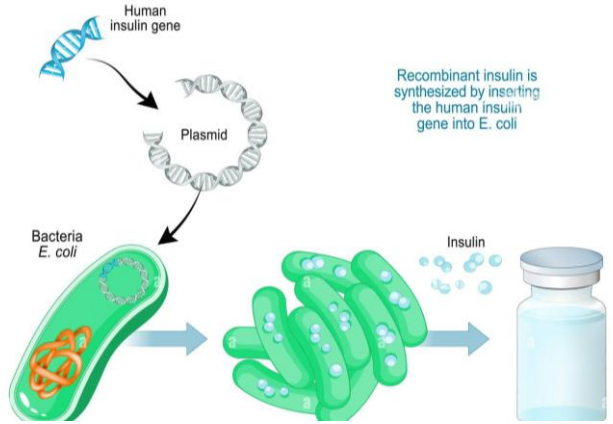
What can we do with sticky ends?

- For instance, we can take a plasmid (circular DNA code, typical of bacteria), **break it to expose sticky ends**, and **insert a new gene** decorated with **sticky ends** matching those of the plasmid



- Now that bacterium will also **express the new gene**

- This is how **insuline** is produced!



…or we could build a computer!

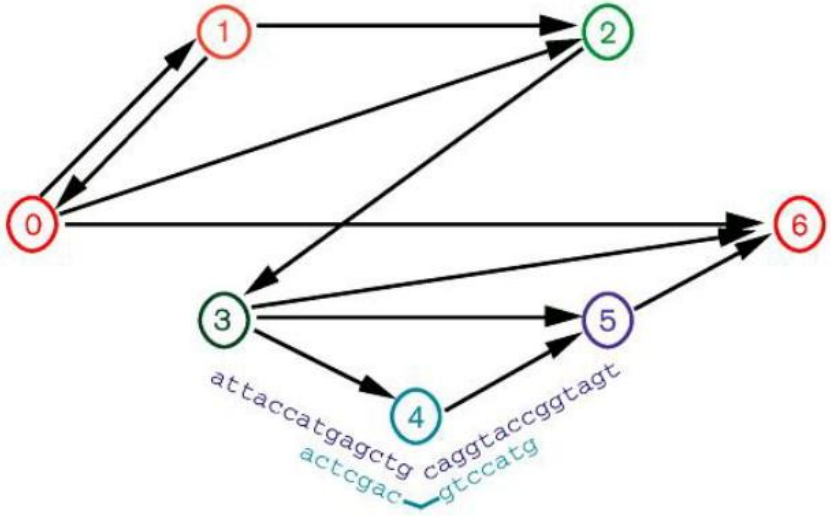
- **Leonard Adleman** (Turing award 2002), computer scientist, co-inventor of the **RSA encryption algorithm**
- He pioneered the **use of DNA to perform computation**
- His computer was used in 1994 to **solve the HPP!** [Adleman, Science 1994]

Adleman’s DNA computing device

- Adleman had the intuition of **encoding pieces of the HPP solution with DNA**
- The idea is that solutions of the HPP will **spontaneously self-assembly in the test tube**
- We can read the result after the self-assembly process is completed

DNA encoding in Adleman's computer

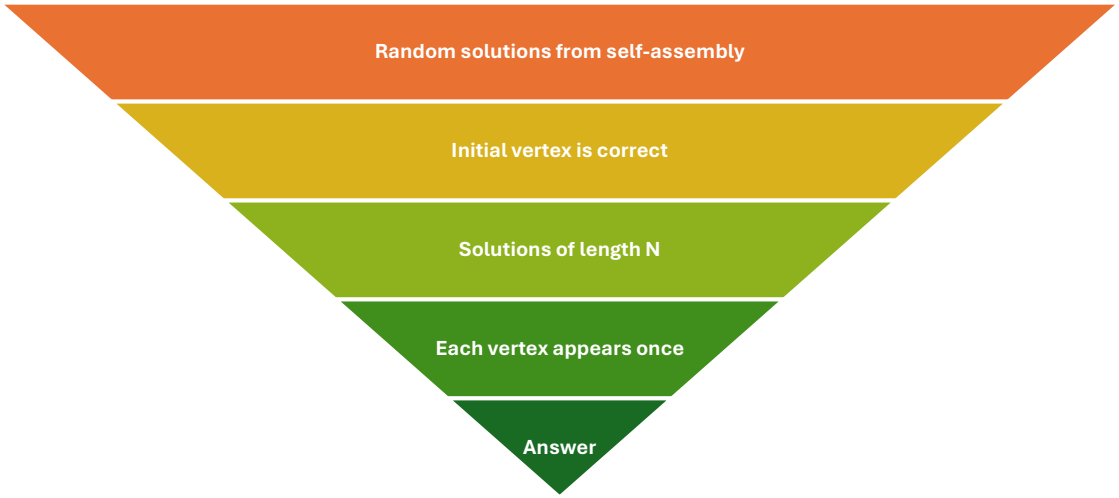
- Each **vertex of the HPP** is represented by a (unique) **20-bases DNA single strand**
  - Why so long? To diversify the encoding as much as possible to prevent wrong assemblies



- The **connection/edge** between the cities/vertices is encoded by a **10-bases strand**

- By putting **plenty of such strands** in a test tube we can exploit spontaneous assembly to **explore a massive amount of parallel solutions**

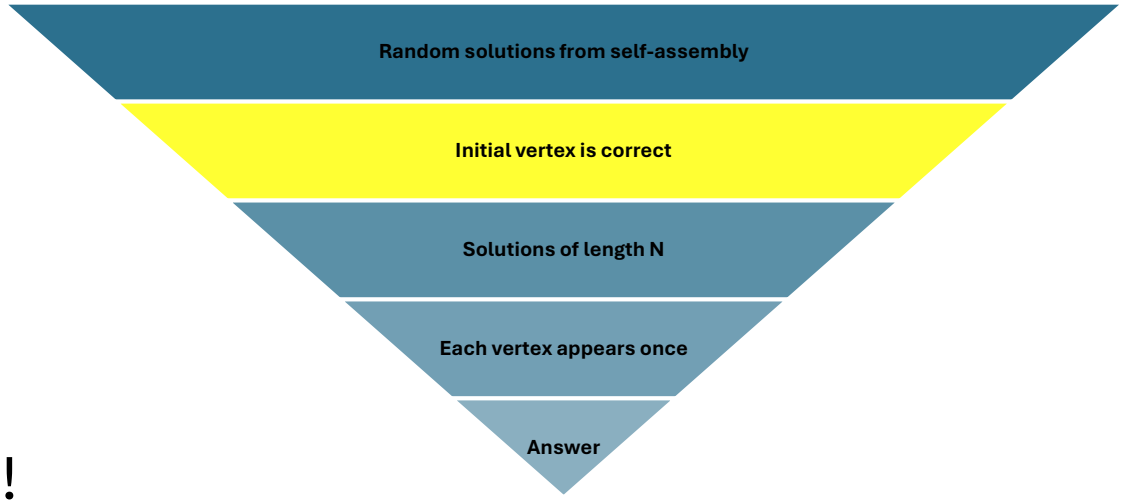
Identifying the solution



After we generated the candidate solutions in the test tube, **to detect whether there is an actual solution** to the HPP we need to perform the following steps:

1) Drop any residual single-strand DNA
2) Keep only those solutions that **start and end with the given vertex**
3) Keep only the **solutions of length N**, where N is the number of vertices
4) Keep only those solutions where **each vertex appears only once**
5) The remaining DNA double strand is the **solution** of the HPP!

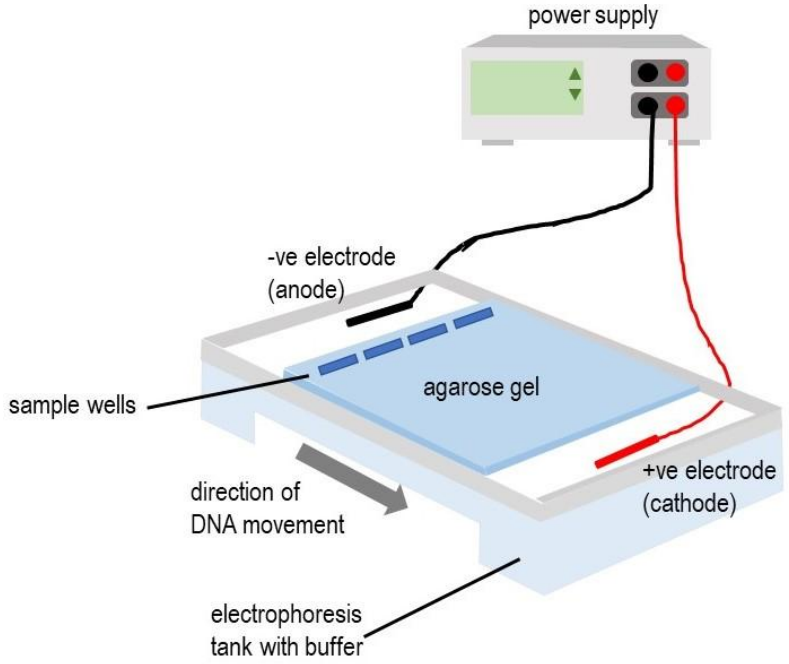
Correct initial vertex

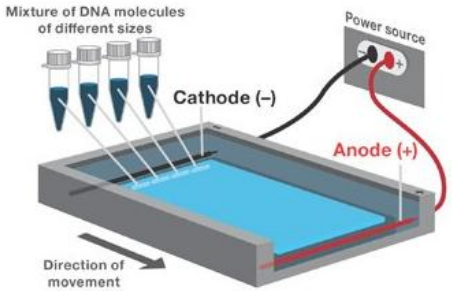


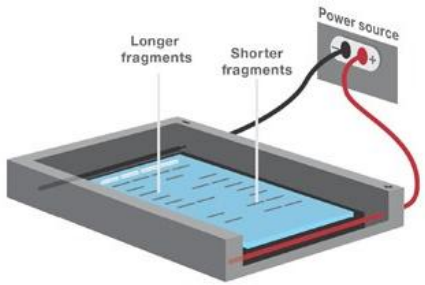
- We use PCR with **a primer that binds to the first vertex**: it will amplify only those sequences that start from the right «city»!
- Any other sequence will basically disappear from the results
- If we want a specific **final vertex**, we can use PCR and **another primer**

Checking the length with gel electrophoresis

- Gel electrophoresis **separates DNA strands according to their size**
- It is based on an **electrophoretic chamber filled with agarose gel**
- **DNA is attracted** by the positive catode
- The gel acts as a sieve: **smaller molecules travel longer distances**



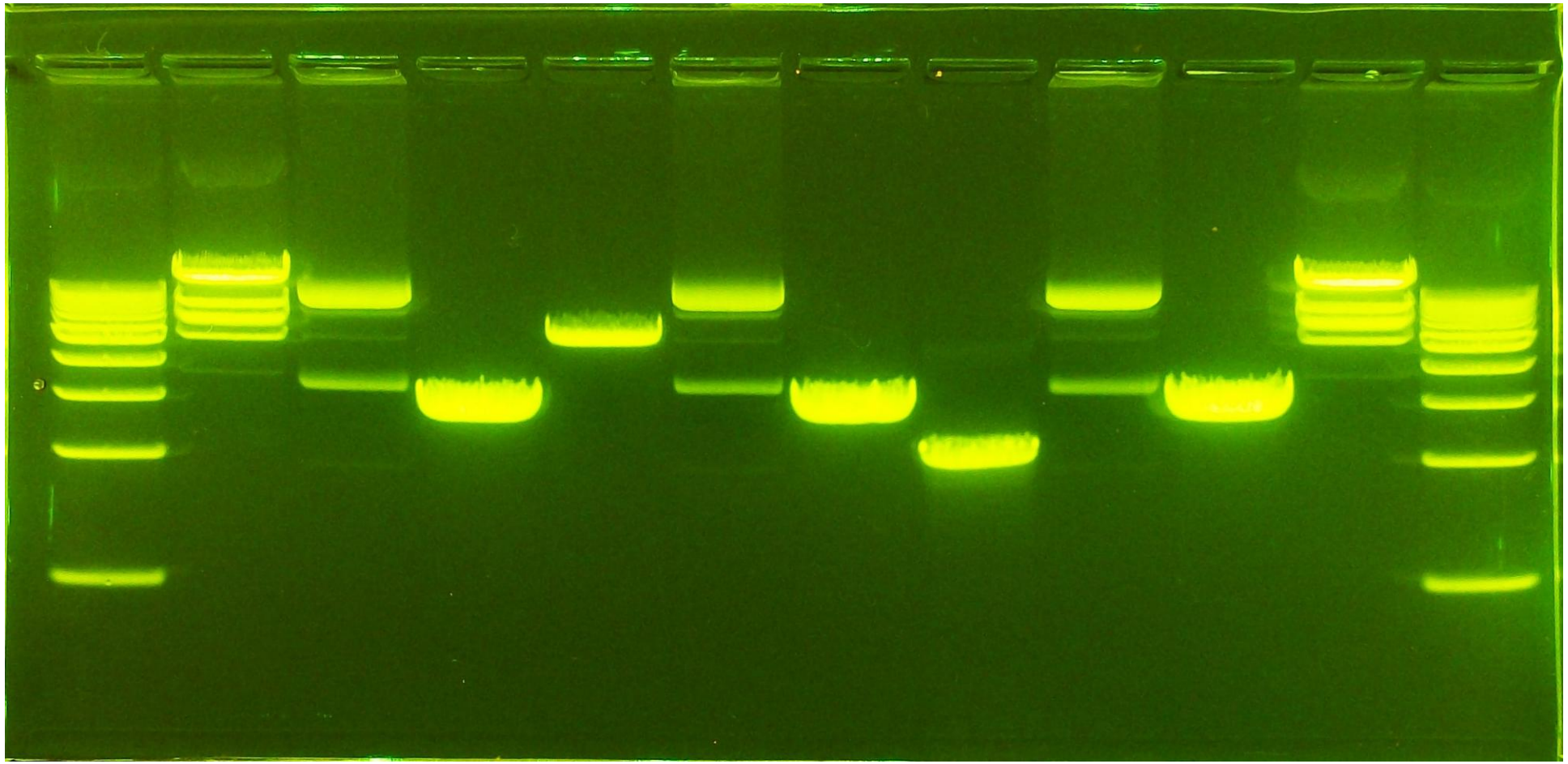




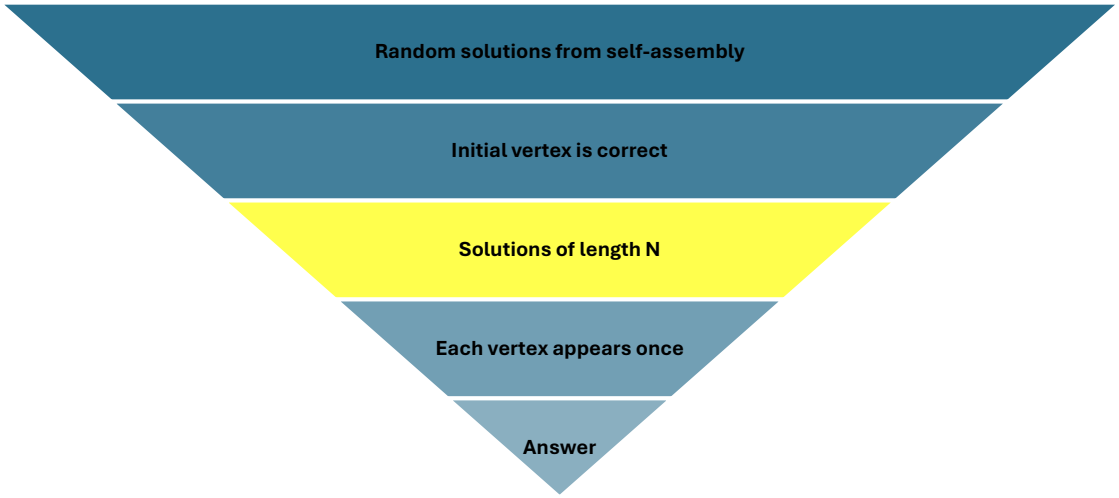
Example of gel electrophoresis experiment

← Experiments →

← Size / # bases →



Actual DNA experiment



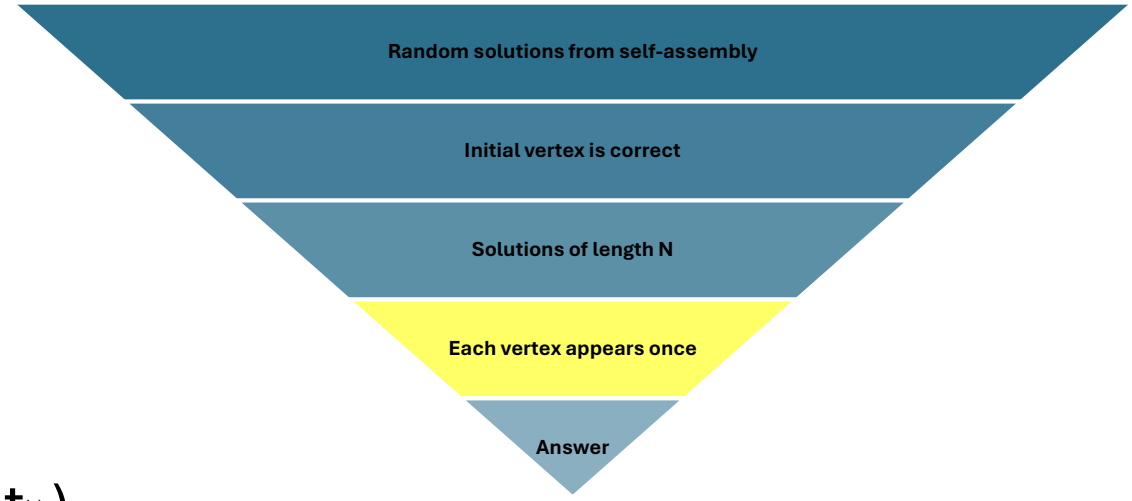
- From [Chang-Muk et al., Molecules Cells 1999]
- **18-mer strands** for vertices, **8 vertices, 14 edges**
- Solution must have exactly 18*8 = **144 bases**

M) Marker of 50 bases

1) Raw DNA material

2) DNA material + ligases

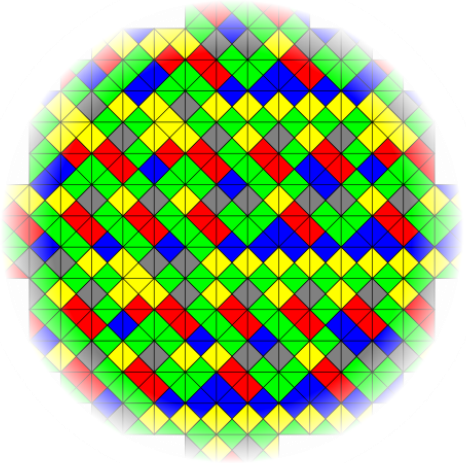
Each vertex appears once

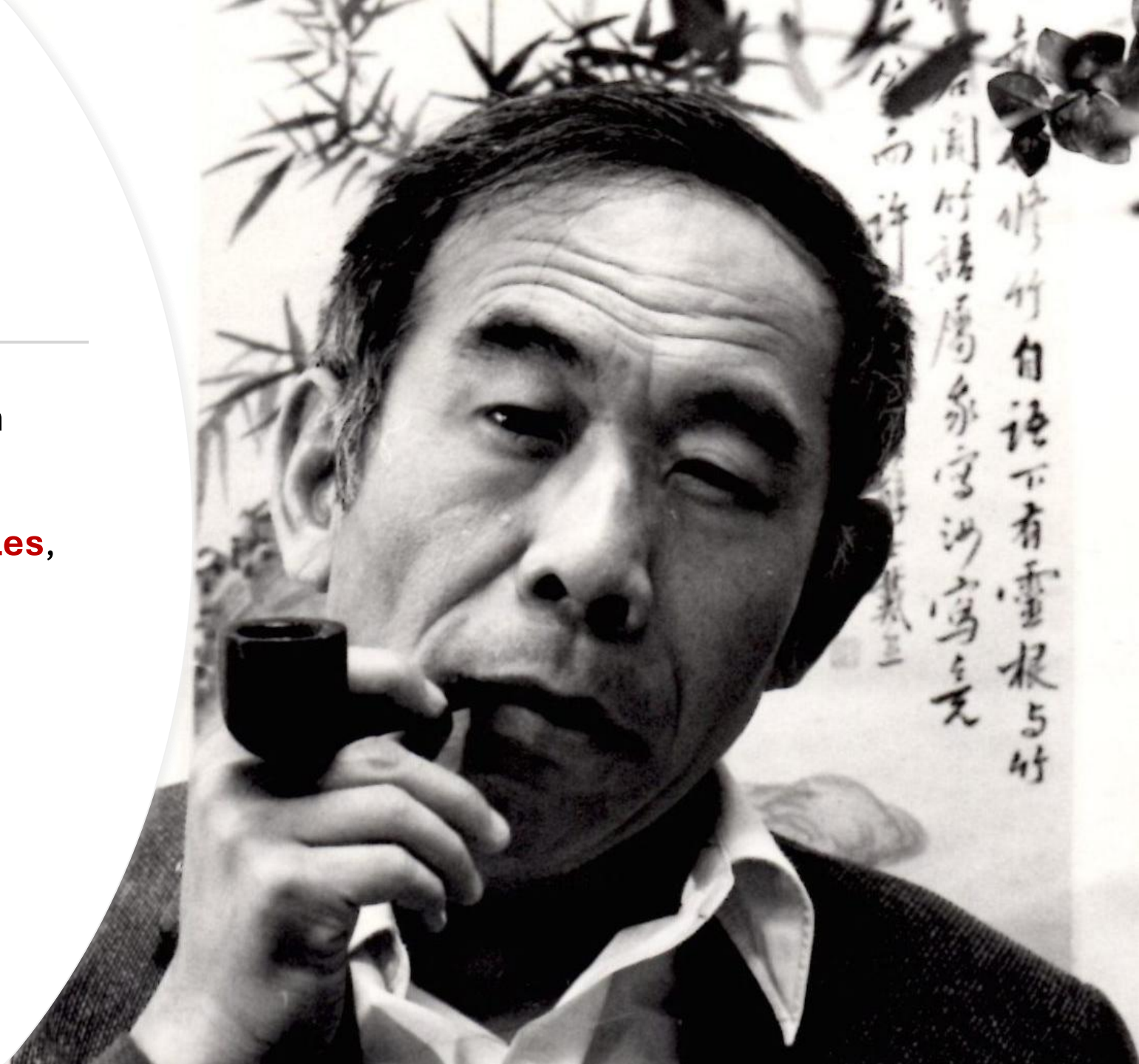


- Adleman used **affinity separation** («the most tedious part of the experiment»)
- He used DNA **«probe» molecules** that can attach to each city/vertex
- The probes were attached to **microscopic iron balls** (~1 micron)
- The idea was to attach the iron balls to each city, one by one, and **transfer the solutions to the next tube** by using magnets
- This way, only solutions that contain all cities can arrive to the final stage
- If the tube is not emtpy, the answer is YES

Hao Wang

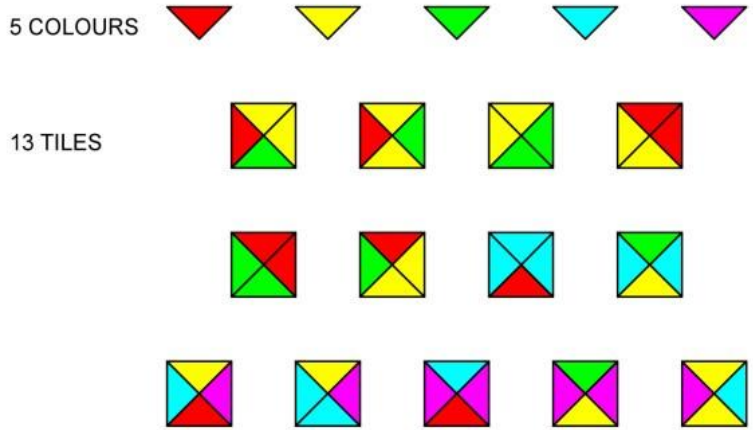
- **Logician**, philosopher, mathematician
- Most important contribution: **Wang Tiles**, **square** tiles with a **color** on each side



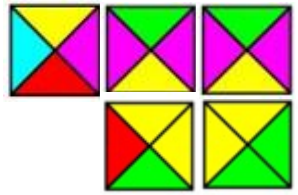


Wang Tiles (WTs)

- **Square tiles**
- Each tile **edge has one specific color**
- There is a fixed amount of **possible colors**
- Tiles are **arranged side by side in a grid**
- **Tiles cannot overlap**
- Each edge **must match the edge of adjacent tiles**
- Tiles have **fixed orientation**
  (i.e., are not rotated nor reflected)



Example:

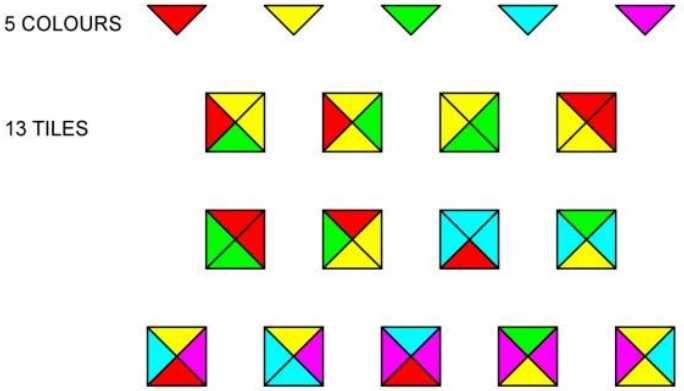


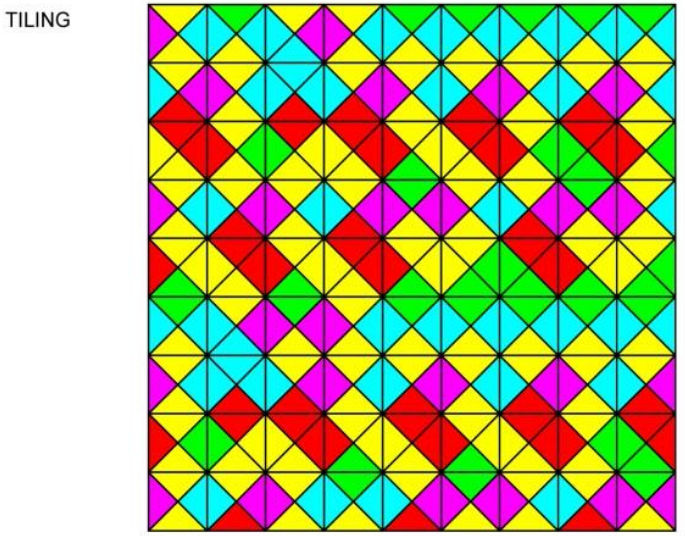
One surprising result of WTs: aperiodicity

- WTs **do not necessarily produce periodic patterns** and tilings

- It was also showed that WTs **do not always tile the plane**

- Actually, the result is even **stronger than that**…





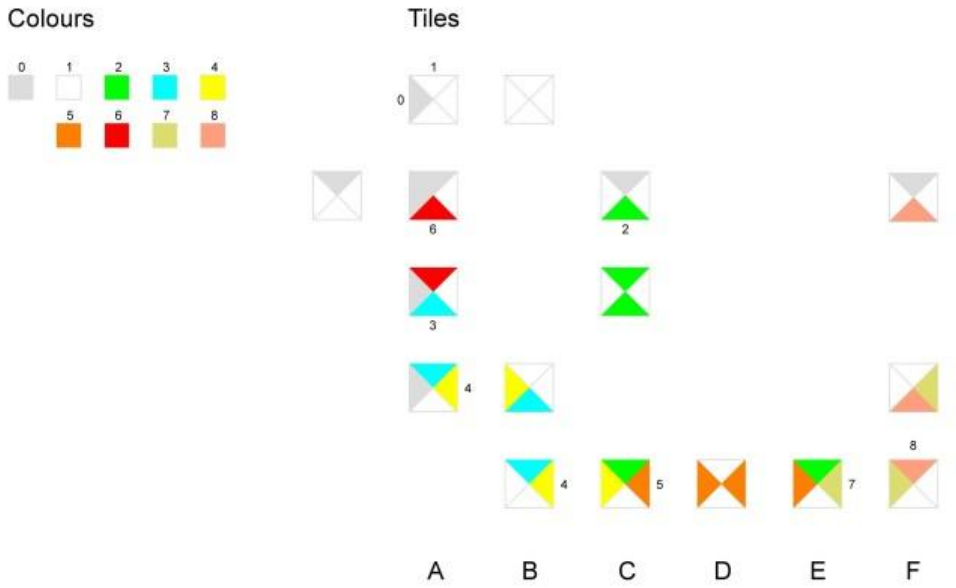
Computational power of WTs

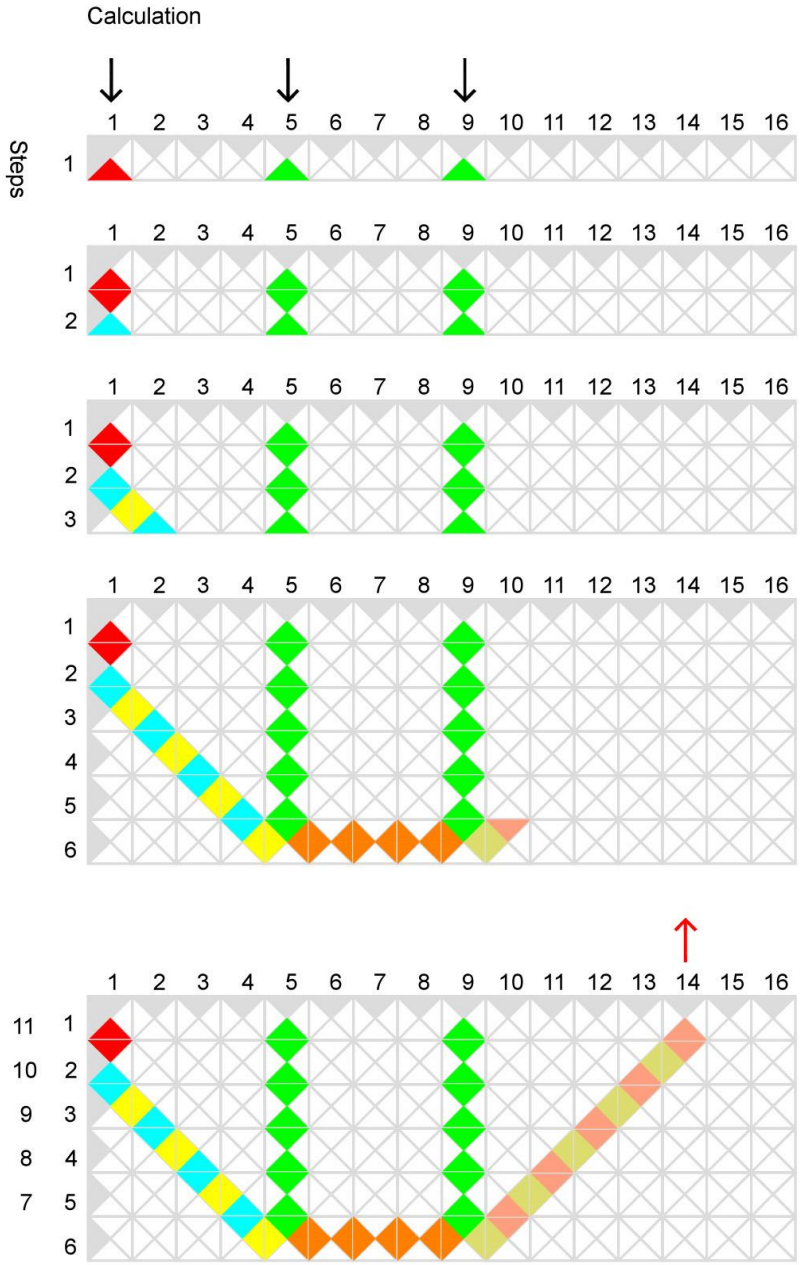
- **There can be no algorithm to decide whether WTs can tile the plane**
- It is possible to **translate a WT into a Turing machine that halts iif the tile set can tile the plane** → this is clearly impossible due to the **undecidability of the halting problem**
- As a matter of fact, **WT can simulate Turing machines**
- Thus, we can **create tiles that perform arbitrary computation!**

Summing two natural numbers with WTs

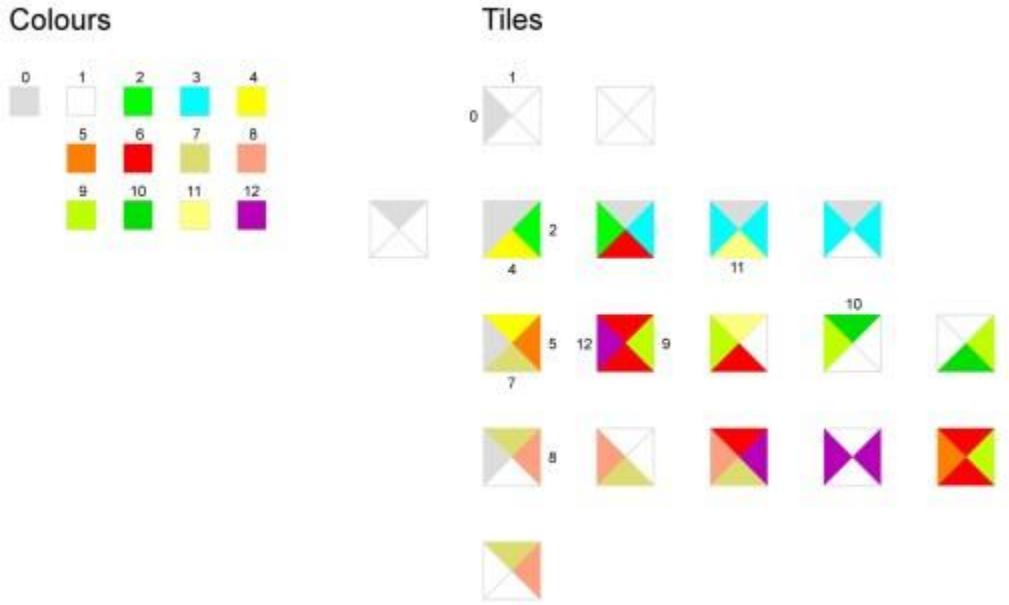
We use green tiles to «set» the numbers to be summed

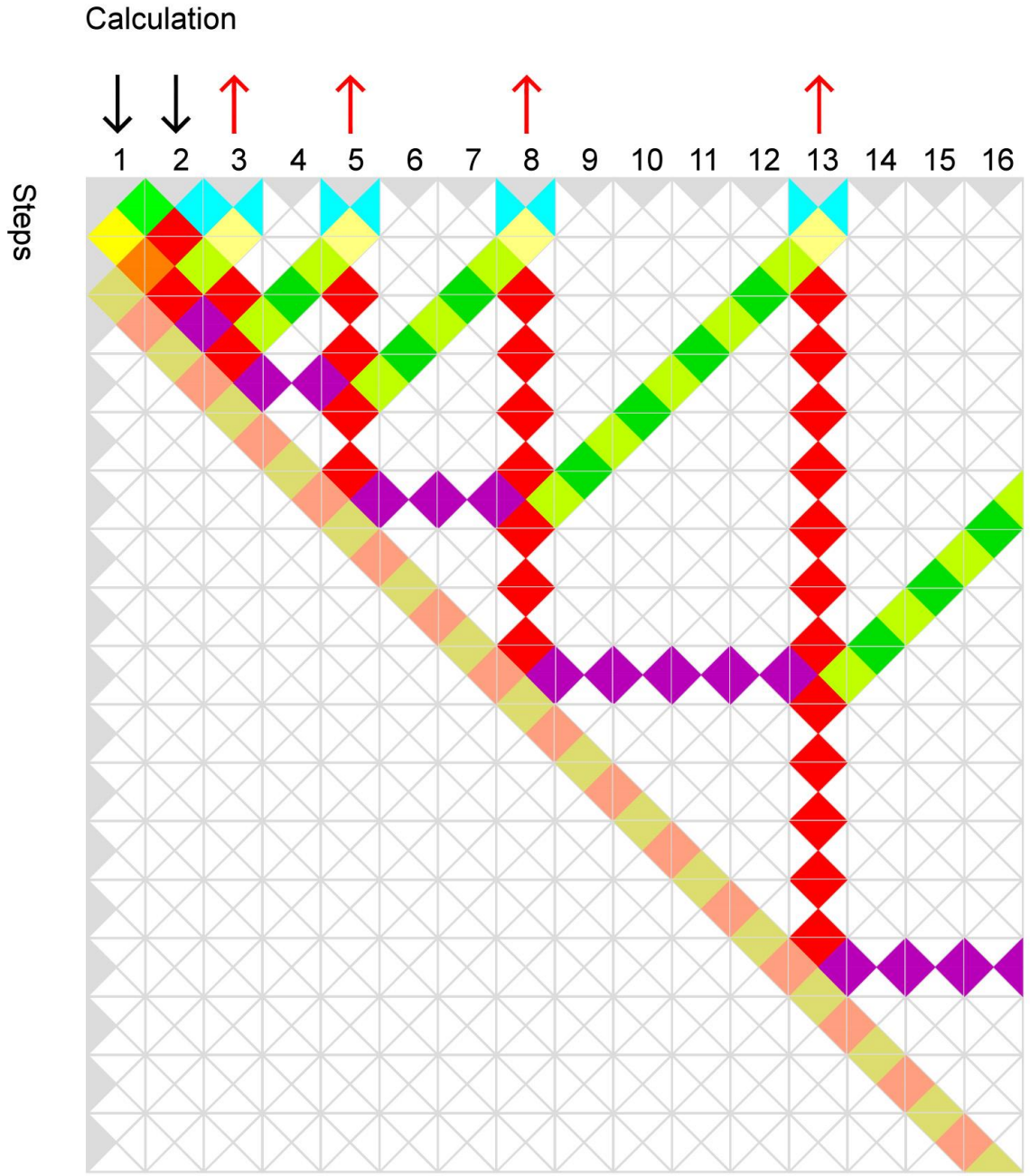
Computation starts from the red tile





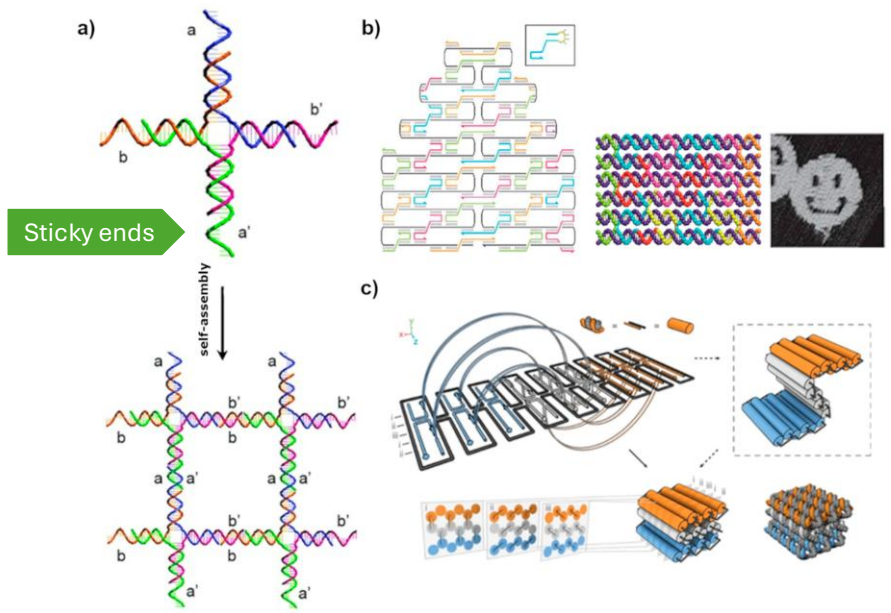
Another example: Fibonacci





DNA WTs

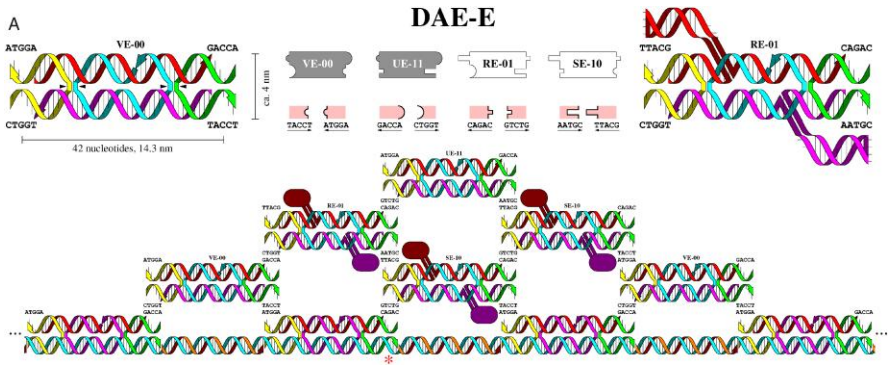
- We can create **DNA WTs with sticky ends**
- **By programming the DNA sequences** on the sticky ends, we can **control the assembly process**
- We can **build all sorts of «images»** and DNA structures on a nano-scale
- Scheme from [Hakker et al., J Biochem Struct Dynam 2013]



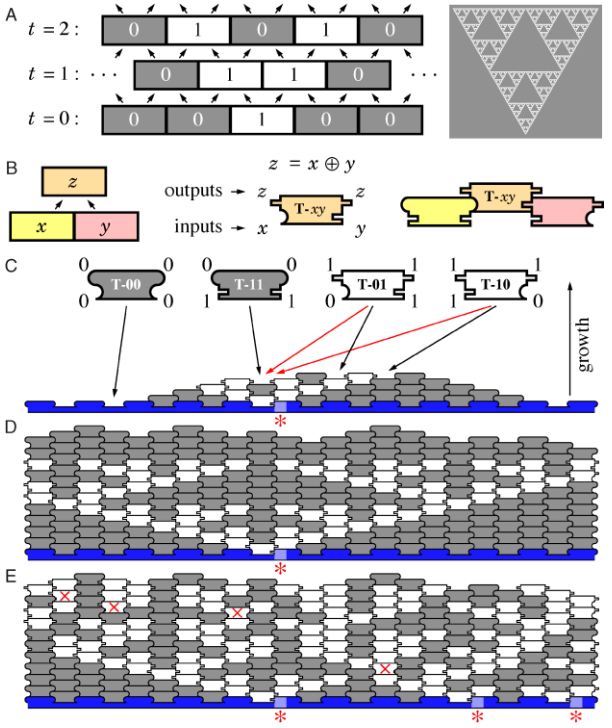
Algorithmic self-assembly of DNA Sierpinsky triangles

WT-based computation

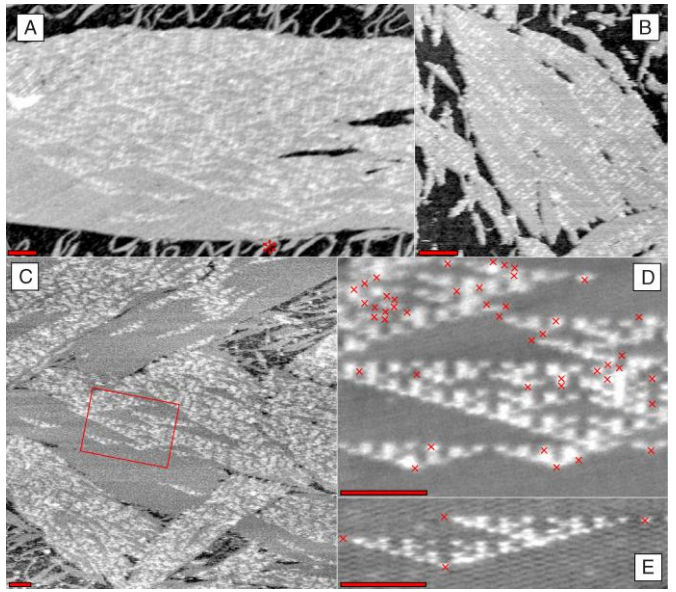
[Rothemund,..., Winfree, PLOS Biol, 2003]



Four tiles are sufficient to implement the rules! →

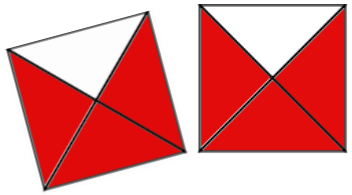


Atomic force microscopy reveals the Sierpinsky triangles →

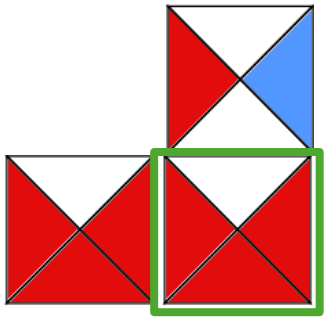


Implementing a NDTM with DNA WTs

- Consider again **DNA-based Wang tiles**
- Imagine that you **choose a temperature** for the experiment such that the binding of WTs is **stable only if at least two sticky ends are attached**

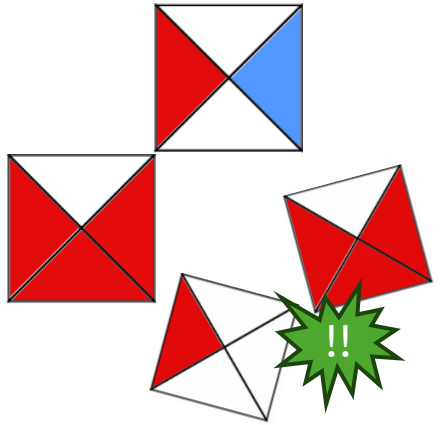


Not stable



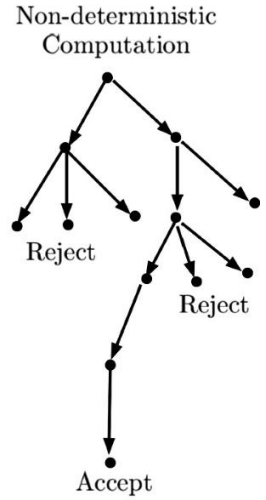
Stable

- Finally, design the sticky ends in such a way that **more than one tile type can attach** in the same position: the tiles will (stochastically) **fight for that position**



Non-determinism is the key

- We can create **billions of DNA WTs** and let them «run» in the test tube
- Thanks to non-determinism in the attachment, **we can explore multiple different computations at once**
- It can be shown that, with the right concentration, the probability of exploring **all possible computations** becomes close to 1
- That is exactly what we needed to implement a NDTM! One of the tile structures could be the **solution of our problem**



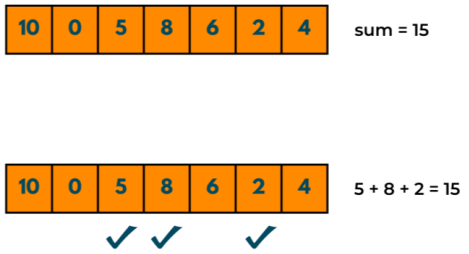
Some technical issues: input and output

- We need to provide an **arbitrary input** to the DNA system: we call that a **seed**

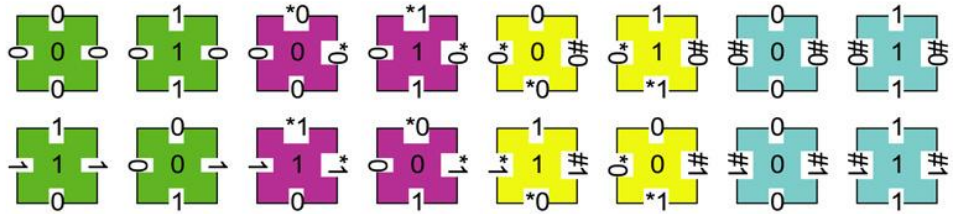
- We also need some sort of **«final tile»** that will be attached only **in the case of successful execution**
  - Possibly, engineered in such a way that we can **easily probe it**
  - One option: a tile decorated with iron or **gold nanoparticles**

One example: SubsetSum

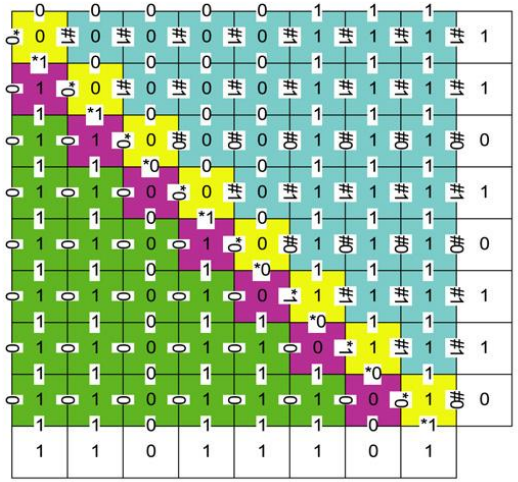
- Given a **sequence of numbers** B and a **value** v, find the subset of B such that the **sum** of the numbers equals v
- NP-complete problem
- **Brun solved SubsetSum with a NDTM based on DNA WTs** [Brun, TCS 2008]
- Five groups of tiles (56 DNA tiles in total):
  1) 16 tiles solving numbers **subtraction**, denoted by T_{-}
  2) 4 tiles implementing the **identity** function, denoted by $\mathbb{S}_{X}$
  3) 20 tiles implementing **nondetermistic guesses**, denoted by $T_{?}$
  4) 7 tiles are used to create the seed Γ_{SS}
  5) 9 tiles calculate the **final decision**, denoted by $T_{\checkmark}$
- The **seed** is a **L-shaped DNA crystal** with the binary representations of the **value** v (bottom) and the **input numbers** in B (right-most column)



Subtraction tiles T_{-}

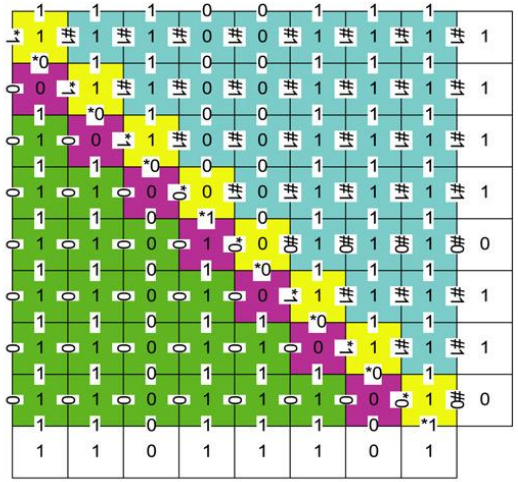


- The problem can be restated as: which subset of values should be **subtracted** from v?
- We need to be able to subtract values



(a)

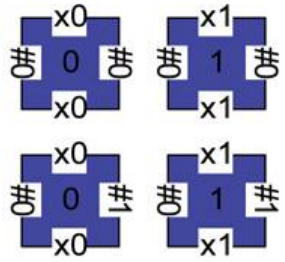
- In (a) we are calculating $221 - 214$
- 11011101 and 01101011, respectively
- The result is correctly 00000111=7



(b)

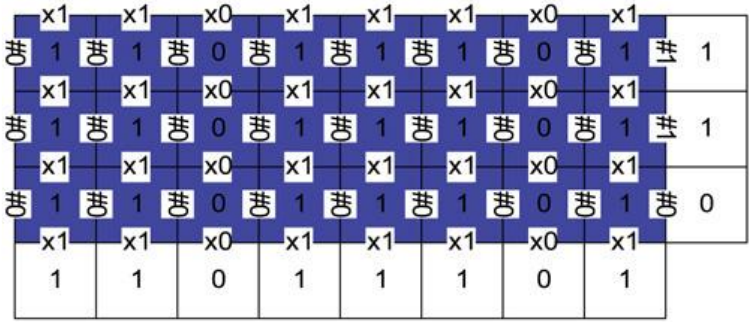
- In (b) we calculate 221-246
- 11011101 and 11110110, respectively
- **This computation fails**
- Failure is denoted by the yellow tile  in the top-left corner with west domain set to 1*
- More about failure later

Identity tiles $\mathbb{S}_X$

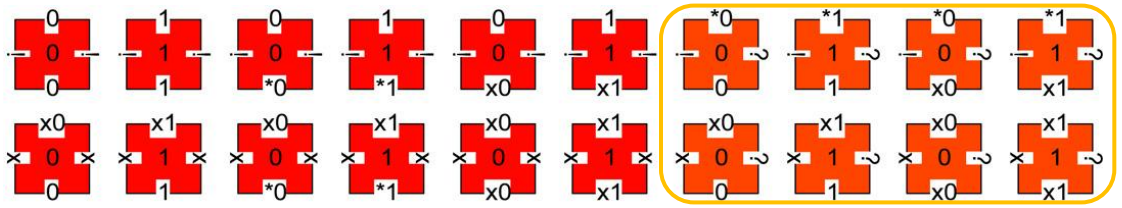


- These tiles just **copy the value** below to the row above
- These are used to **exclude one number** from the set (that is, not subtracted)

- In the example, the result of the previous iteration was 11011101 and that number is **copied** regardless of what is encoded in the seed on the right



Non-deterministic guess $T_?$

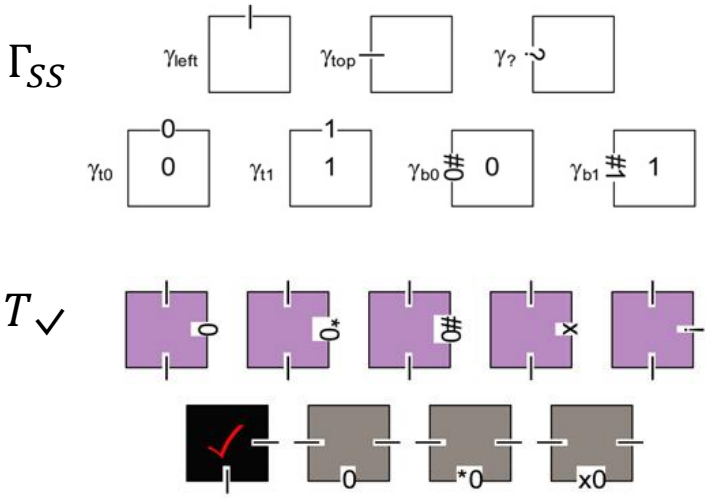


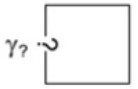
- These tiles implement the **non-deterministic choice** of which numbers in B should be considered for the solution

- Specifically, the tiles inside the orange box **do not have unique east/south binding domains**, thus will attach non-deterministically

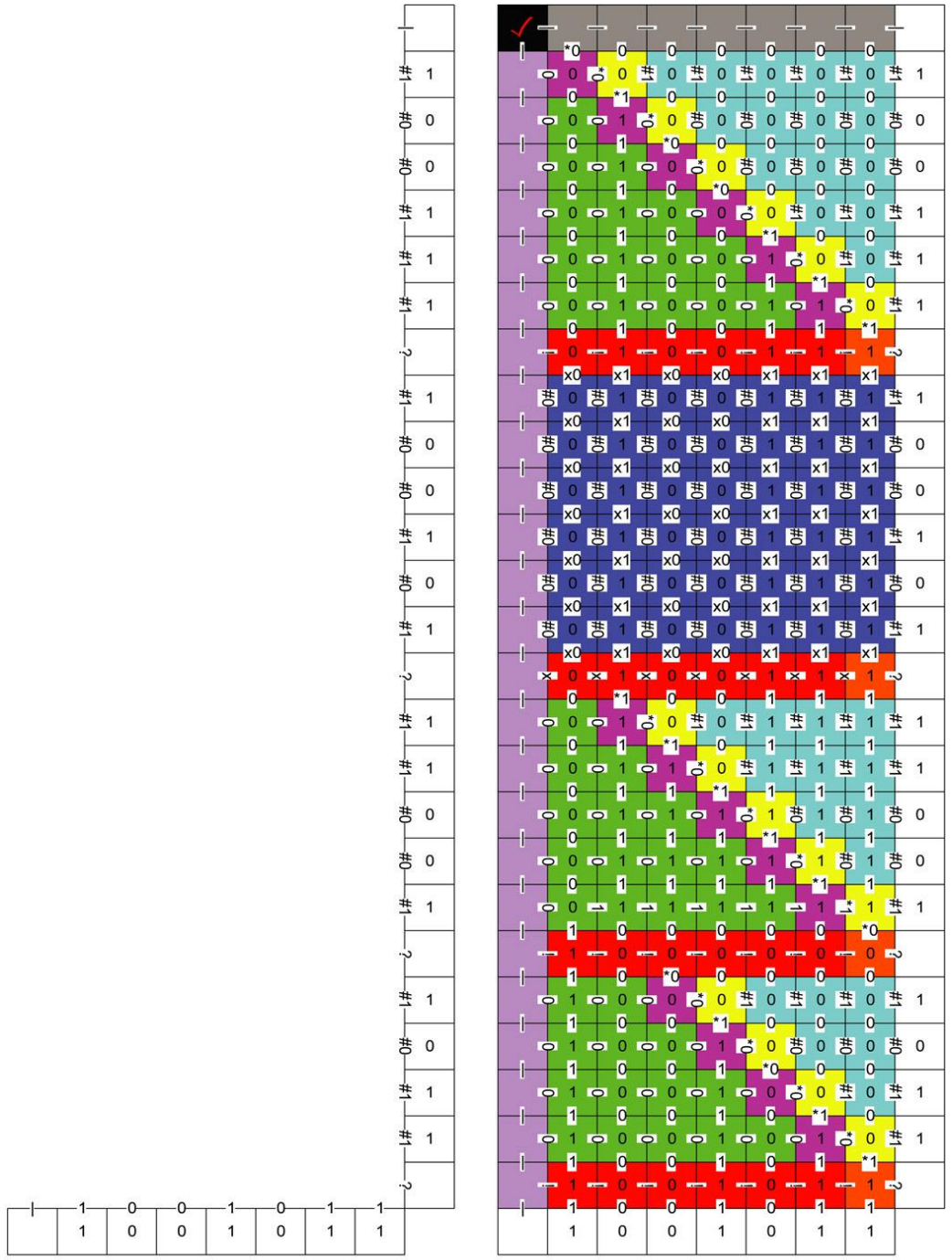
- Thus, one of the two will be **randomly chosen during the self-assembly process** leading to a **non-deterministic computation**
(which is the whole point of this experiment)

Final decision tiles Γ_{SS} and $T_{\checkmark}$

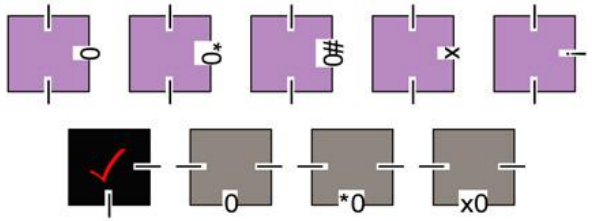


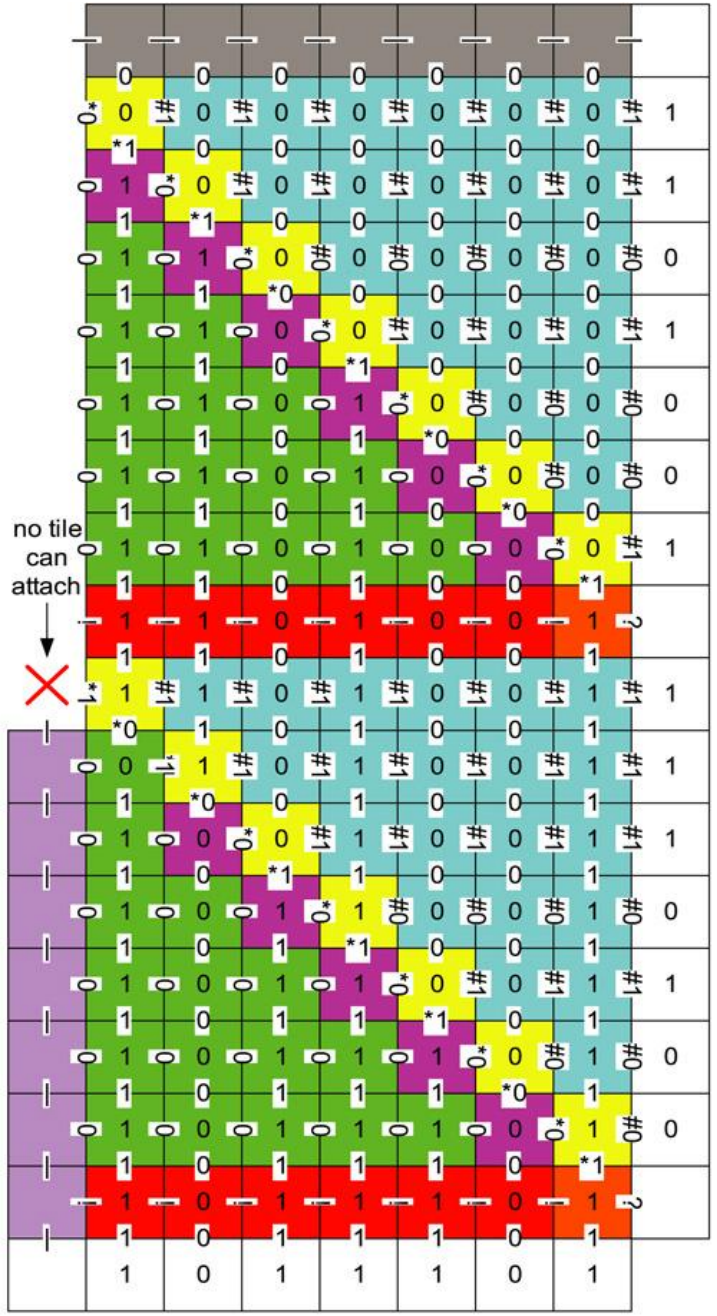
- These tiles are attached at the end of the computation
- Γ_{SS} is used to define the seed, including the tile  which triggers the non-determistic choice
- **The black tile gets attached if and only if the Subset Sum is satisfied**
- If the calculation leads to a negative value then **no more tiles gets attached** and the algorithm halts

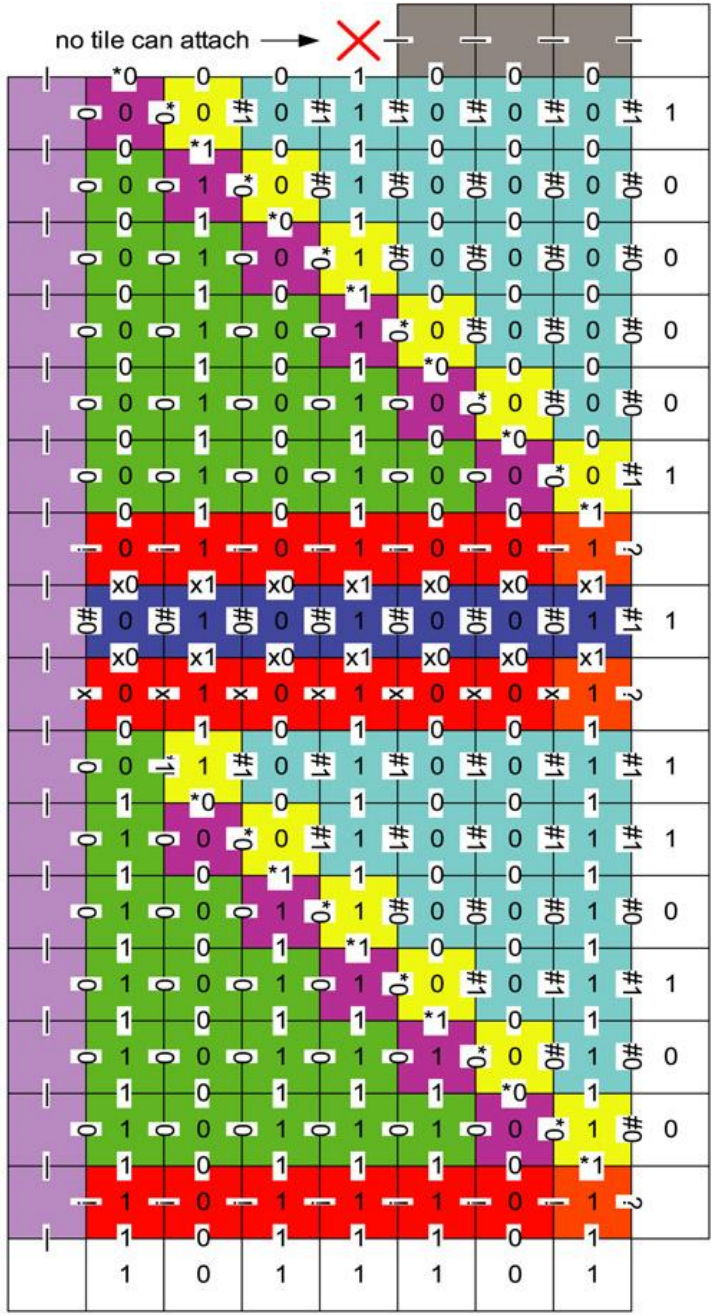
One successful execution



Unsuccesful cases







Other examples of DNA tiles application

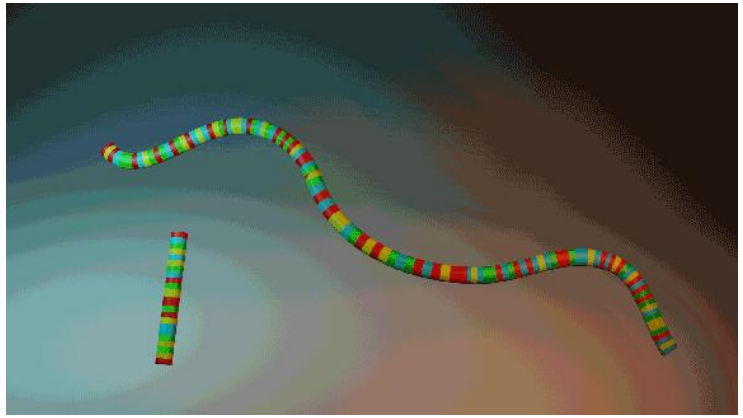
- Timetable problem [Cheng et al., Int J Comput Commun Control 2010]
- Image criptography [Patel et al., Int J Adv Res Engin Tech 2018]
- Knapsack 0-1 [Cui et al., Symposium Neural Networks 2009]
- Maximum clique [Cui et al., Advances in Comp Intell 2009]
- Graph Coloring [Zhang et la., J Comp Theor Nanosci 2009]
- Multidimensional Knapsack [Cheng et al., J Comp Theor Nanosci 2010]
- SAT [Brun, Natural Comput 2012]
- …

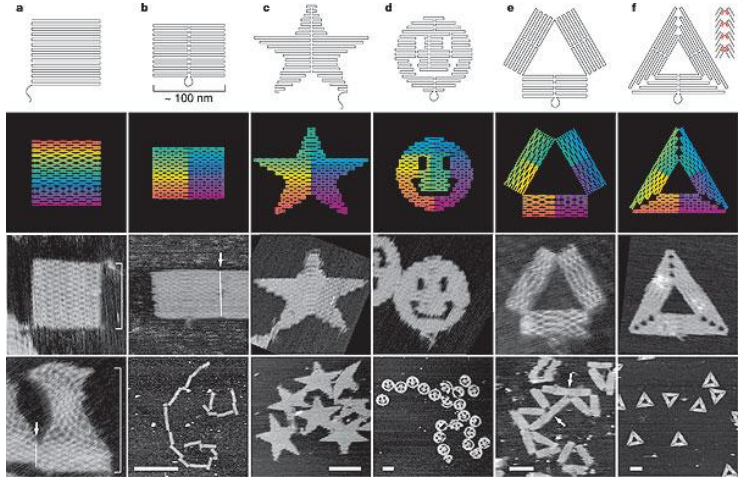
DNA origami

- Nanoscale **folding of DNA** to create arbitrary 2D and 3D shapes

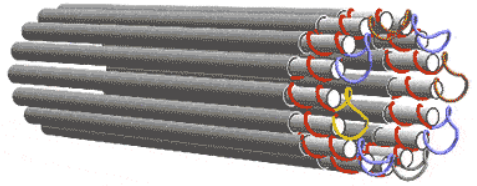
- **A single strand** is folded in a **specific position** by exploiting Watson-Crick complementarity

- Generally, the single strand to be folded is the **genome of the M13 virus** (7249 bases long), while the **DNA «staples» are created in laboratory**

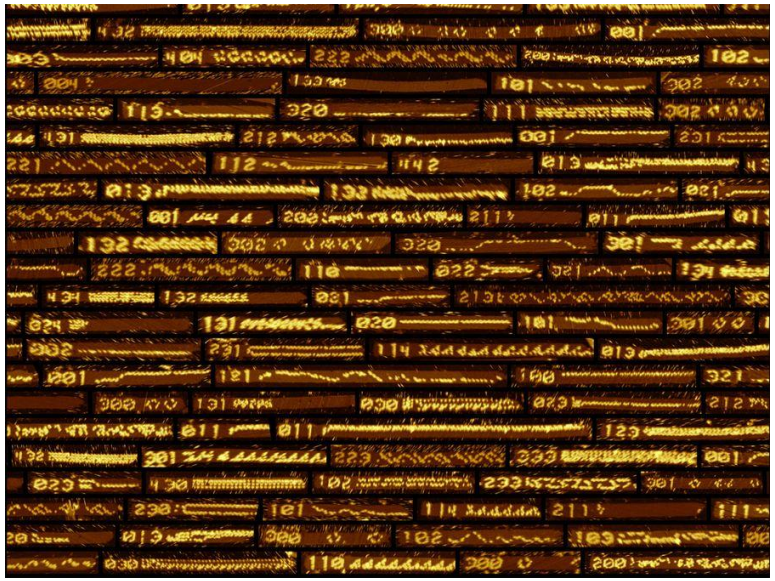




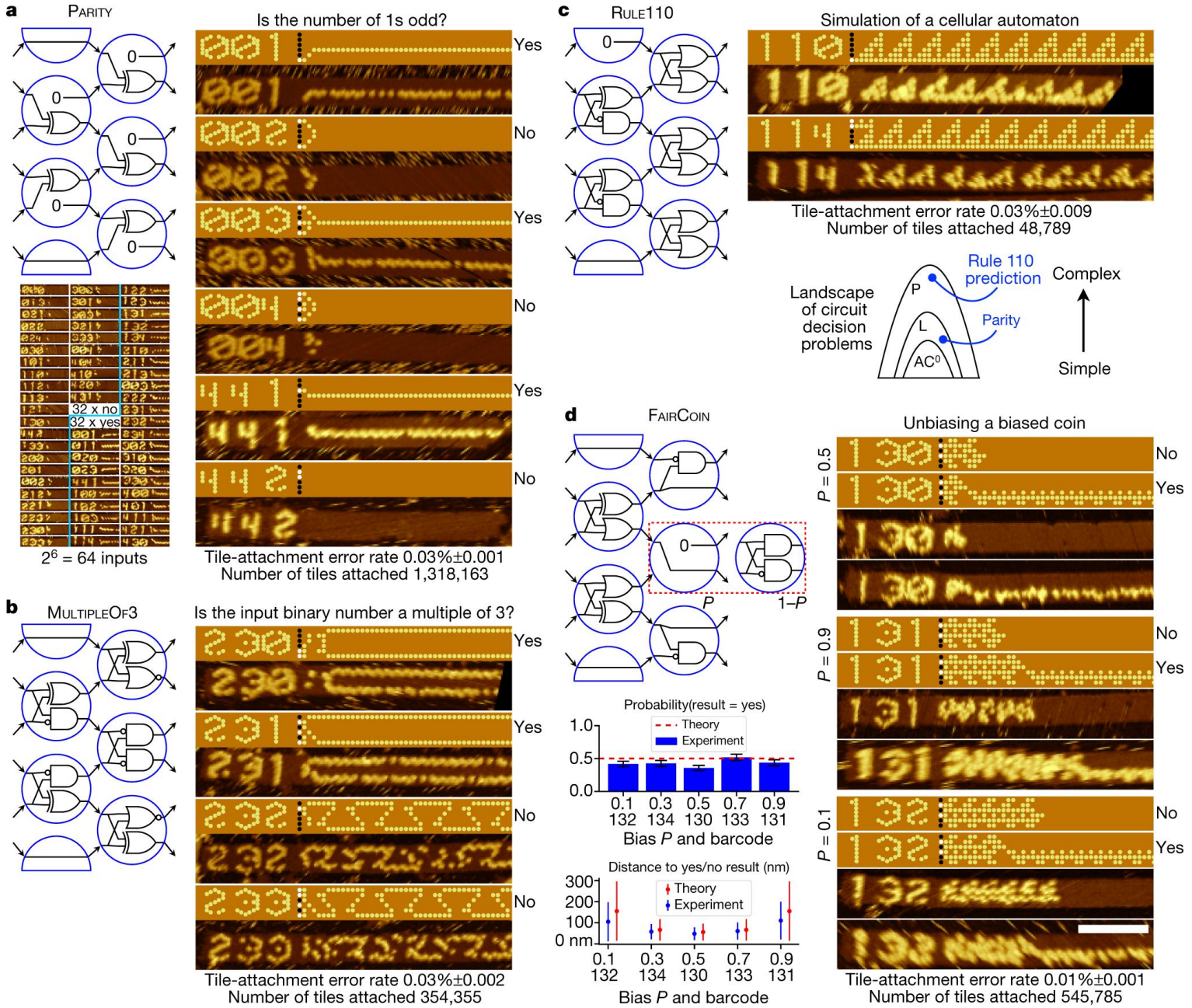
Programmable DNA computers



- [Woods et al., Nature 2019]
- **Seed** (gray): **DNA origami** attached to the the **input (red tiles)**
- Blue, brown, and yellow tiles are **logic gates**
- **1 gate == 4 tiles**
- Overall **355 types of tiles**
- **4 strands of sticky ends** to reduce error
- The algorithm is executed as the system spontaneously grows
- Source code is «compiled» to DNA
- ~1 day for computation to complete



Examples


a
Parity
Is the number of 1s odd?
Yes
No
Yes
No
Yes
No
32 x no
32 x yes
2^6 = 64 inputs
Tile-attachment error rate 0.03%±0.001
Number of tiles attached 1,318,163
b
MultipleOf3
Is the input binary number a multiple of 3?
Yes
Yes
No
No
Tile-attachment error rate 0.03%±0.002
Number of tiles attached 354,355
c
Rule110
Simulation of a cellular automaton
Tile-attachment error rate 0.03%±0.009
Number of tiles attached 48,789
Landscape of circuit decision problems
P
L
AC^0
Rule 110 prediction
Parity
Complex
Simple
d
FairCoin
P
1–P
Probability(result = yes)
Theory
Experiment
1.0
0.5
0.0
0.1 0.3 0.5 0.7 0.9
132 134 130 133 131
Bias P and barcode
Distance to yes/no result (nm)
Theory
Experiment
300
200
100
0 nm
0.1 0.3 0.5 0.7 0.9
132 134 130 133 131
Bias P and barcode
Unbiasing a biased coin
P = 0.5
No
Yes
P = 0.9
No
Yes
P = 0.1
No
Yes
Tile-attachment error rate 0.01%±0.001
Number of tiles attached 545,785

Payload releasing nanobots

nature

Explore content | About the journal | Publish with us

nature › letters › article

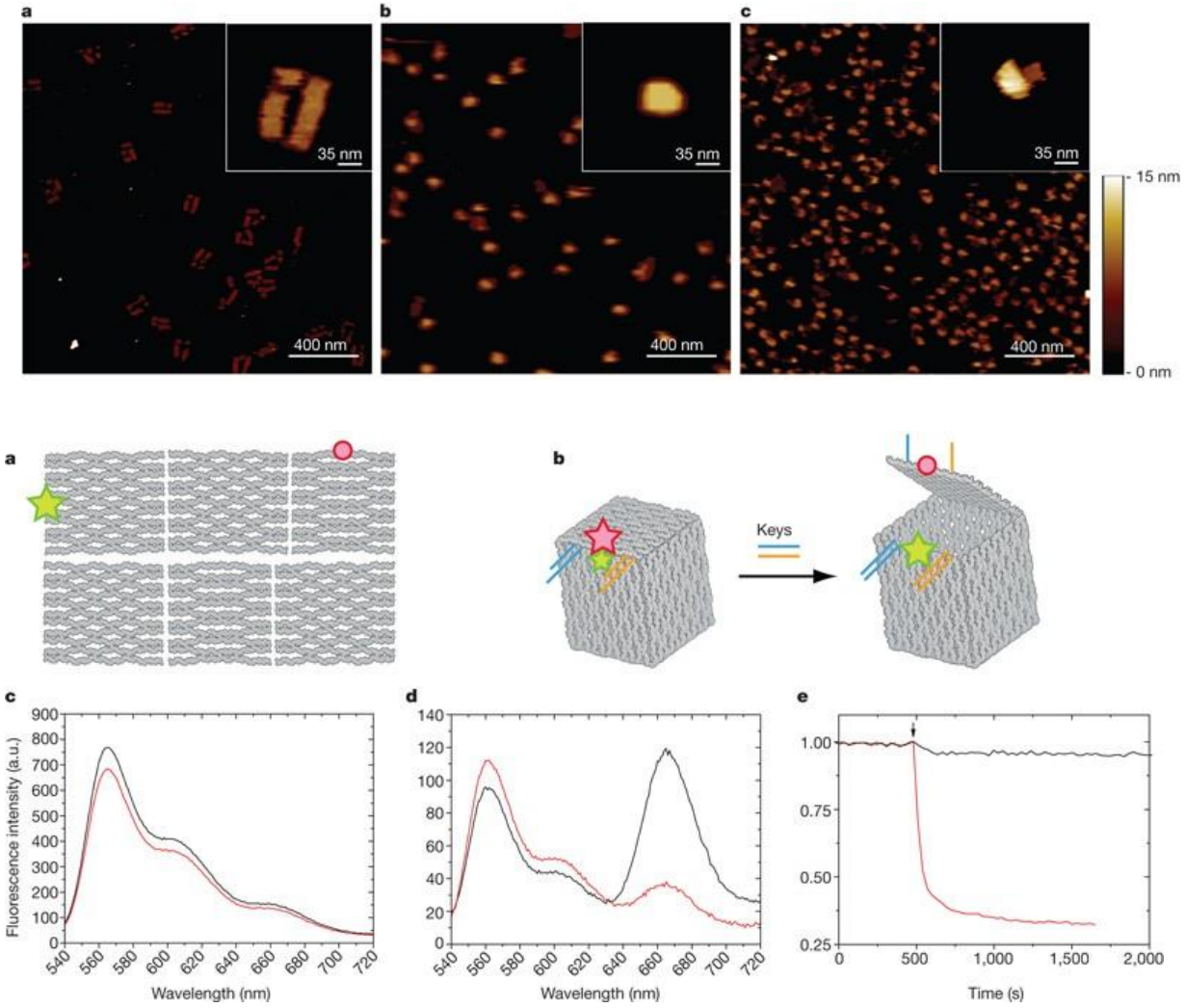
Letter | Published: 07 May 2009

Self-assembly of a nanoscale DNA box with a controllable lid

Ebbe S. Andersen, Mingdong Dong, Morten M. Nielsen, Kasper Jahn, Ramesh Subramani, Wael Mamdouh, Monika M. Golas, Bjoern Sander, Holger Stark, Cristiano L. P. Oliveira, Jan Skov Pedersen, Victoria Birkedal, Flemming Besenbacher, Kurt V. Gothelf & Jørgen Kjems

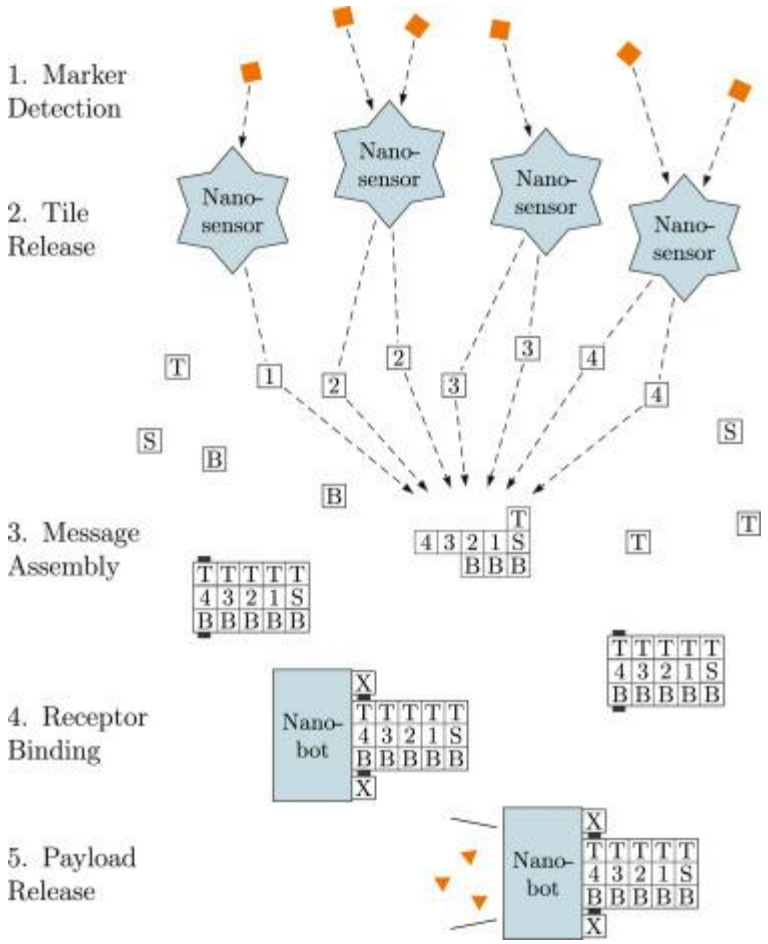
Nature **459**, 73–76 (2009) | Cite this article

33k Accesses | **1530** Citations | **103** Altmetric | Metrics



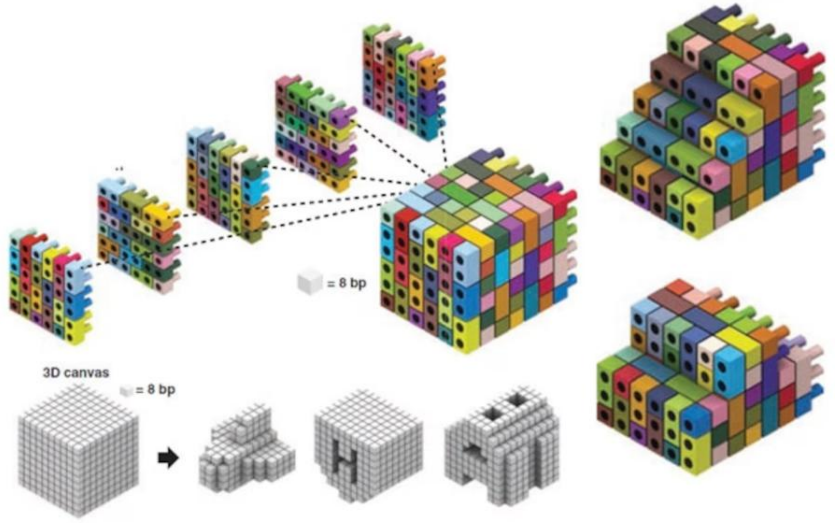
Performing computation in living organisms

- [Lau et al., Nano Communication Networks 2019]
- Nanosensors «feel» the presence of biomarkers and **release WT**
- **The WT assemble** to form a **message**
- The message is received by a more **complex nanobot** that releases a paylod (say, a drug)



From tiles to bricks

- We can also build **«LEGO bricks» with DNA** which **spontaneously assembly**
- We can **program the sticky ends** to produce virtually any **3D shape**
- [Ke et al., Science 2012]



Conclusion

- DNA is a powerful molecule: strong, regular, programmable, excellent for both storage and computation
- It provides a means to create (programmable) nano-artifacts, which spontaneously generate through self-assembly
- We can create nano-computers, operating even in living organisms able to solve NP-complete problems thanks to non-determinism and massive parallelism